Test Plan

Proyecto 2

Portada

Campo	Valor
Proyecto	Verificación del UART 16550 con UVM
Integrante 1	Wilbert Alonso Román Rodríguez
Integrante 2	Gerald Blanco Sáenz
Integrante 3	Leonardo Enrique Pérez Sandoval
Versión del plan	#1
Fecha	Jueves 5 de noviembre del 2026
Repositorio Git	`https://github.com/GeraldBS/Proyecto-2-Verificaci-n-funcional`
Etiqueta Git	entrega1

1. Alcance

Campo	Valor
Versión del RTL verificada	El RTL y la especificación vienen del proyecto uart16550 de OpenCores, espejo freecores/uart16550, commit 2b0ad80 (22 de noviembre de 2004).
Configuración de compilación	Bus de datos de 32 bits (DATA_BUS_WIDTH_8 sin definir), dirección de 5 bits, selección de byte con wb_sel_i y registros Debug presentes y simulador Synopsys VCS R-2020.12-SP2
Frecuencia de reloj en simulación y rango de DL soportado	Reloj de 50 MHz. Se soporta DL de 1 a 65535; por defecto se aleatoriza en 1 a 16 (cubre un 95 %) de 17 en adelante posee probabilidades más bajas, para limitar el tiempo de simulación (un carácter con DL = 65535 es 4 096 veces más largo que con DL = 16). DL = 0 es un valor legal (el UART no transmite ni recibe) y se genera con 1 % por defecto (+UART_DL_ZERO_PCT), igual que en la sección 6.15.
Funcionalidad fuera de alcance y justificación	Variante de bus de 8 bits (DATA_BUS_WIDTH_8) fuera de alcance. Es opcional y se prioriza cerrar la cobertura de la configuración obligatoria de 32 bits.

2. Documentos de referencia

Documento	Versión	Enlace o ubicación
UART IP Core Specification	Rev. 0.6	doc/UART_spec.pdf
RTL del UART 16550 (OpenCores)	commit 2b0ad80 (22/11/2004)	rtl/; espejo github.com/freecores/uart16550
Enunciado del proyecto UVM: UART 16550	versión del 16/10/2026	proyecto_uvm_uart16550.html
Tutorial de instanciación del DUT	-	uart16550_proyecto_uvm/TUTORIAL.md
UVM 1.2 Class Reference y User's Guide	1.2	-
IEEE 1800-2017, SystemVerilog LRM	2017	IEEE
Wishbone Specification	Rev. B.3	opencores.org/howto/wishbone
Synopsys VCS User Guide y Coverage Technology User Guide (URG)	R-2020.12-SP2	-

3. Interpretación de la especificación

3.1 UART 16550

El UART 16550 implementa un protocolo de comunicación entre dos dispositivos independientes, de forma tal que la comunicación se realiza en serie. El UART es el intermediario entre un procesador y un dispositivo por comunicación serial. El UART posee 2 memorias FIFO, una centrada en el envío de datos llamada TX y la otra en la recepción llamada RX. La CPU habla mediante el bus Wishbone leyendo y escribiendo en registros.

Por el otro lado, tiene los pines serie (STX_PAD_O y SRX_PAD_I) y seis señales de módem. Además tiene un pin de interrupción (INT_O) con el que avisa a la CPU cuando necesita atención.

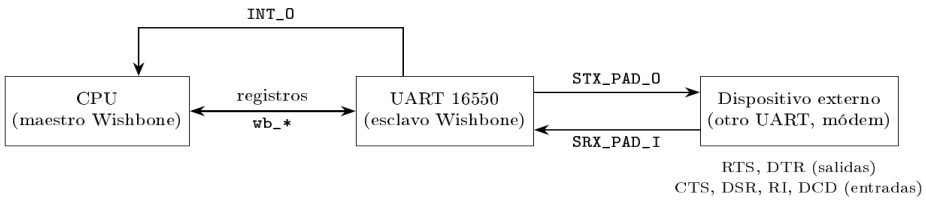


Figura 1. El UART como intermediario entre la CPU y el mundo serie.

3.2 Bloques

- Wishbone (wb_): Medio por el cual la CPU accede a los registros, recibe lecturas y escrituras.
- Divisor Latch (DLL, DLM) y generador de baudios: Produce los pulsos con los que trabajan TX y RX, los baudios se calculan mediante f/(16*DL), donde f es la frecuencia del reloj. Con DL igual a cero el UART queda deshabilitado: no transmite ni recibe.
- Line Control Register (LCR): Define formato, como número de bits de datos, paridad, bits de parada, break y bit DLAB que accede al divisor.
- FIFO TX y registro de desplazamiento TX (THR, STX_PAD_O): CPU escribe datos en la FIFO, TX toma uno cuando el registro está vacío y lo envía bit por bit.
- FIFO RX y registro de desplazamiento RX (RBR, SRX_PAD_I): Proceso inverso de TX, recibe primero en el registro bit por bit, luego guarda en FIFO RX.
- FIFO Control Register (FCR): Control de las FIFO, las limpia y elige nivel de disparo.
- Line Status Register (LSR): Informa el estado de la línea: dato listo, errores (overrun, paridad, trama, break) y si el transmisor está vacío.
- Lógica de interrupciones (IER, IIR, INT_O): Fuentes de interrupción y prioridades. IER habilita cada fuente e IIR indica cuál es la de mayor prioridad pendiente.
- Lógica de señales de módem (MCR, MSR): Maneja salidas RTS y DTR, lee entradas CTS, DSR, RI y DCD.

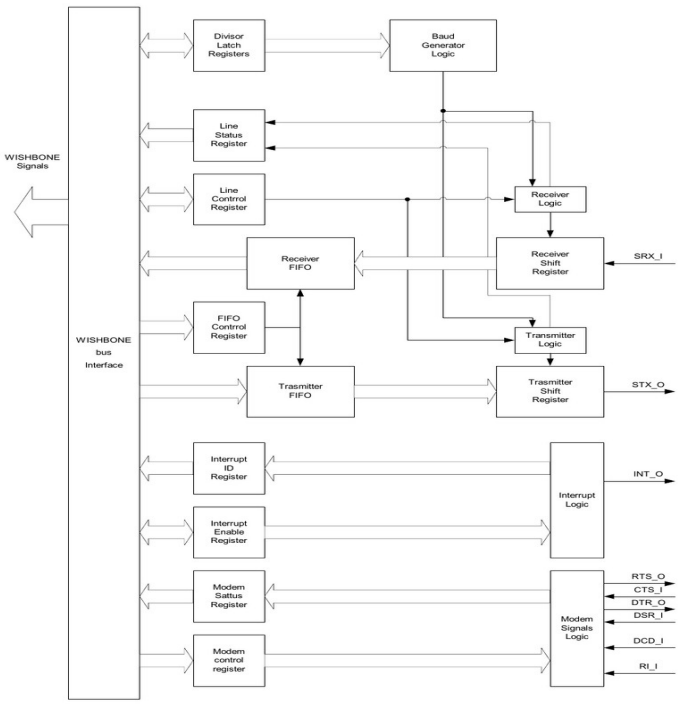


Figura 2. Diagrama de bloques del núcleo UART (Figura 1 de la especificación, Rev. 0.6).

3.3 Proceso de transmisión y recepción

CPU escribe en THR y el dato entra a la FIFO TX, que entrega los caracteres en el orden en que llegaron. Cada vez que el registro de envío queda libre y la FIFO tiene datos, toma el más antiguo y lo envía por STX_PAD_O. LSR[5] (THRE) se pone en 1 cuando la FIFO queda vacía y LSR[6] (TEMT) cuando además el registro de envío terminó. Cada bit dura 16 pulsos del generador de baudios, es decir 16×DL ciclos de reloj. La trama que sale por la línea comienza con un bit de inicio en 0, luego los datos con el menos significativo primero, la paridad si está habilitada y por último 1, 1.5 o 2 bits de parada en 1 (1.5 solo con caracteres de 5 bits y 2 con 6 a 8).

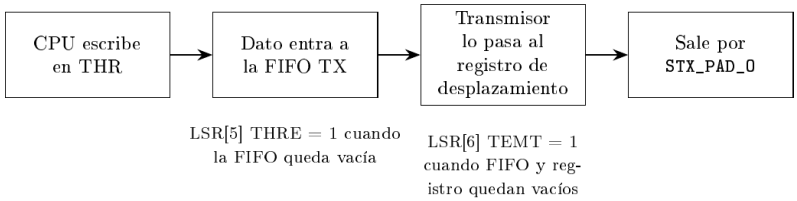


Figura 3. Camino de un carácter transmitido.

En la recepción, la línea está en uno mientras espera. Al detectarse el bit de inicio, se reciben los bits de datos, se revisan la paridad y el bit de parada y, por último, el dato pasa a la FIFO RX.
Los errores que puede detectar el receptor son cuatro: paridad incorrecta, bit de parada inválido (error de trama), línea en 0 durante un carácter completo (break) y overrun, que ocurre cuando llega un carácter nuevo con la FIFO llena y se pierde.

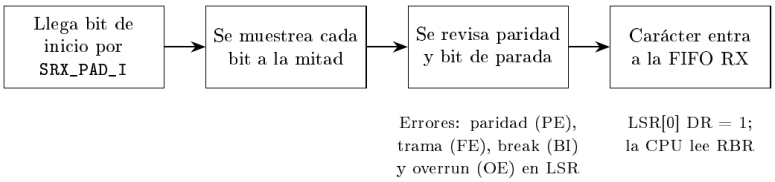


Figura 4. Camino de un carácter recibido.

3.4 Interrupciones

El UART avisa a la CPU con INT_O. Cada fuente se habilita con un bit de IER y IIR indica cuál es la interrupción pendiente de mayor prioridad.

Prioridad	Interrupción	Cuándo ocurre	Qué la limpia
1	Estado de línea	Error de paridad, trama, overrun o break	Leer LSR
2	Dato disponible	La FIFO RX llega al nivel de disparo	Leer RBR hasta bajar del nivel
2	Timeout	Hay datos en la FIFO RX y pasaron 4 tiempos de carácter sin que entre ni salga ninguno	Leer RBR
3	THR vacío	La FIFO TX queda vacía	Escribir THR o leer IIR
4	Estado del módem	Cambio en CTS, DSR, RI o DCD	Leer MSR

3.5 Señales de módem y modo loopback

DTR y RTS son salidas controladas por MCR. CTS, DSR, RI y DCD son entradas que se leen en MSR. Con el bit de loopback en MCR, la salida del transmisor se conecta internamente a la entrada del receptor y las salidas de módem se conectan a las entradas (DTR a DSR, RTS a CTS, Out1 a RI y Out2 a DCD).

3.6 DLAB

Divisor Latch Access Bit, es el bit 7 de LCR, es un interruptor que cambia que registros se ven.

- DLAB = 0: RBR lee, THR escribe y la dirección 1 es IER.
- DLAB = 1: la dirección 0 es DLL y la 1 es DLM.
- Las direcciones 2 a 6 no cambian.

3.7 Ambigüedades entre la especificación y el RTL

Una fila por cada ambigüedad o discrepancia entre la especificación y el RTL (sección 3.5 del enunciado).

ID	Tema	La especificación dice	El RTL hace	Comportamiento que modelamos	Justificación
AMB-01	Registro scratch en la dirección 7	La sección 5 dice que el scratch fue eliminado y la tabla 4.1 no lista la dirección 7	Existe, es un registro de lectura y escritura de 8 bits en la dirección 7; empieza en 0 y la lectura devuelve el último valor escrito.	Registro de lectura y escritura de 8 bits en la dirección 7, con valor 0x00 después del reset	El historial del RTL muestra que el registro scratch se agregó a propósito (revisión 1.28, 10/12/2001), aunque la especificación dice que fue eliminado. Como lo que se simula es el RTL, modelamos el registro tal como existe.
AMB-02	Lectura de MCR	MCR es solo escritura (W en la tabla 4.1)	La lectura de la dirección 4 devuelve 0x00, porque MCR no está en el case de lectura.	MCR es de solo escritura y su lectura devuelve 0x00. Para conocer su valor real se usa la puerta trasera como dice TUTORIAL.md, sección 6.	La especificación solo declara MCR de escritura y no define qué devuelve una lectura. El RTL devuelve 0x00, así que ese es el valor que esperamos al leerlo. El scoreboard no espera recuperar el dato escrito.
AMB-03	Polaridad de rts_pad_o, dtr_pad_o y de MSR [7:4]	El pin sale invertido respecto al bit de MCR: con el bit en 0 el pin está en 1, y con el bit en 1 el pin está en 0. MSR [7:4]: cada bit es el complemento de la entrada de módem (CTS, DSR, RI, DCD)	Los pads están directos, sin invertir. MSR [7:4]: en loopback toma los bits de MCR; fuera de loopback toma el valor de los pads sin invertirlos.	rts_pad_o o dtr_pad_o es el complemento del bit de MCR. MSR [7:4] es el complemento de la entrada de módem (CTS, DSR, RI, DCD) y en loopback es igual al bit de MCR correspondiente.	La especificación es el documento normativo. En loopback el RTL coincide con la especificación, pero fuera de loopback rts, dtr y MSR [7:4] tienen la polaridad opuesta. Se reporta como posible error del DUT.
AMB-04	Condición de TERI (MSR [2])	Se activa cuando RI cambia de bajo a alto según uart_spec.pdf	Se activa con cualquier cambio de RI (ambos flancos)	TERI (MSR [2]) se activa solo con el flanco de subida de ri_pad_i (de 0 a 1)	La especificación define TERI con un solo sentido de cambio, y es el documento normativo del proyecto. El RTL difiere: usa un XOR con el valor del ciclo anterior, así que se activa con ambos flancos.
AMB-05	wb_sel_i con más o menos de un bit activo (modo 32 bits)	wb_sel_i sirve para recibir y devolver datos de 8 bits en el bus de 32	Con wb_sel_i sin exactamente un bit activo, el UART toma el registro 0 (o el 4, según la dirección); al escribir usa dat [7:0] como dato. La lectura devuelve 0, pero la acción igual ocurre	Los accesos válidos usan un solo bit activo en wb_sel_i, o 1111 solo para los registros Debug (direcciones 8 y 12). Con un sel raro no se revisa el dato que se lee, porque nadie dice cuál debería ser.	La especificación no define este caso, así que el dato devuelto no es un error. Se modela la acción del RTL para no desincronizar el scoreboard, y se mantiene con baja probabilidad, como exige el enunciado.
AMB-06	1.5 y 2 bits de parada, y qué parada verifica el receptor	LCR [2] = 0: 1 bit de parada. LCR [2] = 1: 1.5 bits si el carácter es de 5 bits, y 2 bits si es de 6, 7 u 8. El receptor siempre revisa solo el primer bit de parada.	Transmisor: genera 1 bit de parada si LCR [2] = 0, 1.5 si LCR [2] = 1 con 5 bits, y 2 en los demás casos. Receptor: lee solo el primer bit de parada, a la mitad del bit, y no revisa ningún segundo bit.	Transmisor: 1, 1.5 o 2 bits de parada según LCR [2] y la longitud del carácter. Receptor: lee solo el primer bit de parada (si vale 0, error de trama) y no revisa el segundo	La especificación y el RTL coinciden. Se documenta porque el enunciado lo exige y porque define las aserciones del agente serie.
AMB-07	Lecturas y escrituras con DLAB = 1	Solo dice que, con DLAB = 1, las direcciones 0 y 1 dejan de dar acceso a RBR/THR e IER y pasan a ser el divisor; nada sobre los demás registros	Las acciones que se hacen al leer LSR, MSR e IIR exigen DLAB = 0. Con DLAB = 1, leer LSR, MSR o IIR devuelve el valor pero no limpia nada.	Comportamiento del RTL: con DLAB = 1, leer LSR, MSR o IIR no cambia el estado; el RAL y el scoreboard no esperan limpieza en esos accesos	La especificación no lo cubre, así que se modela el RTL para que los accesos aleatorios con DLAB = 1 no desincronicen al scoreboard
AMB-08	Layout y reset de Debug 2 (dirección 12)	Sección 4.11: MCR en los bits 18:17 y FCR en los bits 23:19.	uart_debug_if arma {8'b0, fcr, mcr, rf_count, rstate, tf_count, tstate}: MCR de 5 bits en 21:17 y FCR (los bits 7:6, 2 bits) en 23:22. Con el reset FCR vale 2'b11, así que Debug 2 lee 32'h00C00000.	Debug 2 con el layout del RTL (MCR en 21:17, FCR en 23:22) y reset 32'h00C00000. Debug 1 coincide con la especificación (reset 32'h00031060).	La propia tabla de la especificación es inconsistente: asigna 2 bits a un valor de 5 bits (MCR) y 5 bits a uno de 2 bits (FCR), así que no puede ser el layout real. El RTL es consistente con los registros que copia. Como el RAL compara Debug 2 en hw_reset, usar el layout de la especificación daría una falla falsa.

4. Lista de características

ID	Característica	Fuente (sección de la especificación)	Prioridad (alta, media, baja)
F-WB-01	Transferencias Wishbone de lectura y escritura con acuse (ack)	IO ports	Alta
F-INT-07	int_o refleja la interrupción pendiente	IO ports (INT_O)	Alta
F-RST-02	Reset en medio de tráfico: el UART vuelve al estado inicial sin quedar colgado	IO ports (WB_RST_I)	Alta
F-REG-01	Mapa de registros y tipo de acceso, incluido el scratch en la dirección 7 (AMB-01) y MCR de solo escritura (AMB-02)	Registers list	Alta
F-REG-04	Registros que comparten dirección según DLAB; efectos de acceder con DLAB = 1 (AMB-07)	Registers list; LCR bit 7	Media
F-TX-02	Escritura en THR: los datos entran a la FIFO de transmisión y salen en orden	Registers list (THR)	Alta
F-RX-02	Lectura de RBR: los datos salen de la FIFO de recepción en orden	Registers list (RBR)	Alta
F-RX-05	Lectura de RBR con la FIFO de recepción vacía	Registers list (RBR); la especificación no define el valor devuelto	Baja
F-INT-06	Habilitación individual de cada interrupción con IER	Interrupt Enable Register	Alta
F-INT-01	Interrupción de estado de línea (prioridad 1)	Interrupt Identification Register	Alta
F-INT-02	Interrupción de dato disponible al llegar al nivel de disparo	Interrupt Identification Register	Alta
F-INT-03	Interrupción de timeout tras 4 tiempos de carácter	Interrupt Identification Register	Alta
F-INT-04	Interrupción de THRE (se limpia al escribir THR o leer IIR)	Interrupt Identification Register	Alta
F-INT-05	Interrupción de estado del módem (se limpia al leer MSR)	Interrupt Identification Register	Alta
F-INT-08	Prioridad entre fuentes de interrupción y bit 0 de IIR (interrupción pendiente)	Interrupt Identification Register	Alta
F-FIFO-01	Nivel de disparo de recepción: 1, 4, 8 o 14 caracteres	FIFO Control Register, bits 7:6	Alta
F-TX-04	Limpiar la FIFO de TX (FCR [2]) no corta el carácter en curso	FIFO Control Register, bit 2	Media
F-RX-03	Limpiar la FIFO de RX (FCR [1]) no corta el carácter en curso	FIFO Control Register, bit 1	Media
F-LCR-01	Longitud de carácter de 5, 6, 7 y 8 bits	Line Control Register, bits 1:0	Alta
F-LCR-02	Bits de parada: 1, y 1.5 con 5 bits o 2 con 6 a 8 (AMB-06)	Line Control Register, bit 2	Alta
F-LCR-03	Paridad habilitada, impar o par	Line Control Register, bits 3 y 4	Alta
F-LCR-04	Paridad stick en 0 o en 1	Line Control Register, bit 5	Media
F-LCR-05	Break control: la salida serie se fuerza a 0	Line Control Register, bit 6	Media
F-TX-01	Transmisión serie: bit de inicio, datos con el bit menos significativo primero, paridad y parada	Line Control Register; Operation	Alta
F-MDM-01	Salidas DTR y RTS controladas por MCR (con su polaridad, AMB-03)	Modem Control Register, bits 0 y 1	Alta
F-MDM-02	Bits Out1 y Out2 de MCR	Modem Control Register, bits 2 y 3	Baja
F-LB-01	Modo loopback: stx_pad_o en 1 y el transmisor conectado al receptor	Modem Control Register, bit 4	Alta
F-LB-02	Loopback de módem: DTR→DSR, RTS→CTS, Out1→RI, Out2→DCD	Modem Control Register, bit 4	Alta
F-ERR-01	LSR [0]: dato disponible	Line Status Register	Alta

ID	Característica	Fuente (sección de la especificación)	Prioridad (alta, media, baja)
F-ERR-02	LSR [1]: error de desbordamiento (overrun); la FIFO queda intacta	Line Status Register	Alta
F-ERR-03	LSR [2]: error de paridad del carácter al tope de la FIFO	Line Status Register	Alta
F-ERR-04	LSR [3]: error de trama (stop inválido)	Line Status Register	Alta
F-ERR-05	LSR [4]: break (línea en 0 durante un carácter completo)	Line Status Register	Alta
F-ERR-06	LSR [5]: FIFO de transmisión vacía (THRE)	Line Status Register	Alta
F-ERR-07	LSR [6]: transmisor vacío (FIFO y registro de desplazamiento)	Line Status Register	Alta
F-ERR-08	LSR [7]: error de paridad, trama o break dentro de la FIFO	Line Status Register	Media
F-ERR-09	Los bits de error de LSR se limpian al leerlo	Line Status Register	Alta
F-MDM-03	MSR [7:4] refleja las entradas de módem (con su polaridad, AMB-03)	Modem Status Register, bits 7:4	Alta
F-MDM-04	Deltas DCTS, DDSR y DDCD ante cambios de las entradas	Modem Status Register, bits 0, 1 y 3	Alta
F-MDM-05	TERI al cambiar RI de bajo a alto (AMB-04)	Modem Status Register, bit 2	Alta
F-MDM-06	Los deltas de MSR se limpian al leerlo	Modem Status Register	Alta
F-DL-01	Divisor de 16 bits (DLL y DLM) accesible con DLAB = 1	Divisor Latches; LCR bit 7	Alta
F-DL-02	Tasa de baudios = reloj / (16 × DL) en transmisión y recepción	Divisor Latches	Alta
F-DL-03	DL = 0 desactiva la entrada y salida serie; DL máximo = 65535	Divisor Latches	Media
F-DL-04	Escritura del divisor: DLM primero y DLL al final (el contador arranca al escribir DLL)	Divisor Latches	Media
F-REG-03	Registros Debug 1 y Debug 2 (AMB-08)	Debug 1; Debug 2	Media
F-WB-02	Selección de byte con wb_sel_i en el bus de 32 bits	Operation	Alta
F-WB-03	Accesos con wb_sel_i ilegal (AMB-05)	Operation (hueco)	Baja
F-REG-02	Valores de reset de todos los registros	Operation (Initialization); cada registro	Alta
F-RST-01	Reset asíncrono: estado inicial de FIFOs, registros y línea serie	Operation (Initialization); IO ports	Alta
F-RX-01	Recepción serie: muestreo en la mitad del bit	Operation (Initialization)	Alta
F-RX-04	Tolerancia a pequeñas diferencias de baudios	Operation (Initialization)	Media
F-TX-03	Escritura con la FIFO de transmisión llena	Enunciado, sección 4.1	Media
F-REG-05	Reconfiguración de registros con tráfico activo	Enunciado, sección 4.1	Media

5. Matriz de verificación

Una o más filas por cada característica de la sección 4.

ID ítem	Característica	Qué se verifica	Mecanismo (scoreboard, SVA, RAL, cobertura)	Métrica de cierre (coverpoint o aserción)	Plusargs que lo dirigen
V-WB-01	F-WB-01	Cada transferencia Wishbone recibe ack en tiempo acotado, con señales estables hasta el ack	Aserciones A-WB-01 a A-WB-04 y scoreboard	A-WB-01 a A-WB-04 (assert y cover); cg_reg_access.cp_rw	+UART_WB_GAP_MAX, +UART_WB_GAP_ZERO_PCT, +UART_WB_STB_DELAY_PCT, +UART_WB_STB_DELAY_MAX
V-INT-07	F-INT-07	int_o activo solo cuando el modelo prevé una interrupción pendiente	Modelo de interrupciones y aserción A-INT-01	A-INT-01 (assert y cover); cg_int.cp_int_edge	+UART_IER_BIT_PCT, +UART_SW_MODE_W, +UART_SW_DELAY_MAX
V-RST-02	F-RST-02	Estado inicial correcto tras un reset en medio del tráfico	Scoreboard (reinicia el modelo), aserción A-RST-01 y RAL	cg_reset.cp_when, cp_width	+UART_RST_PCT, +UART_RST_WIDTH_MAX
V-REG-01	F-REG-01	Tipo de acceso correcto (R, W, RW) en cada dirección (AMB-01, AMB-02)	RAL (secuencias integradas, exclusiones justificadas)	cg_reg_access.cp_addr × cp_rw	+UART_REG_NOISE_PCT
V-REG-04	F-REG-04	Direcciones 0 y 1 correctas con DLAB = 0 y 1; LSR, MSR e IIR sin efectos laterales con DLAB = 1 (AMB-07)	RAL (mapa DLAB 0/1) y scoreboard	cg_reg_access.cp_dlab × cp_addr × cp_rw	+UART_REG_DLAB_PCT, +UART_REG_NOISE_PCT
V-TX-02	F-TX-02	Los bytes escritos en THR salen por stx en el mismo orden y con el mismo valor	Scoreboard TX	cg_line_cfg.cp_data, cg_tx.cp_level	+UART_TX_BURSTS_MAX, +UART_TX_LEN_W, +UART_DATA_CORNER_PCT
V-RX-02	F-RX-02	Datos recibidos por srx se leen de RBR en orden y sin alteración	Scoreboard RX	cg_rx.cp_data, cp_level	+UART_SER_RX_CHARS_MAX, +UART_DATA_CORNER_PCT, +UART_SW_PARTIAL_PCT
V-RX-05	F-RX-05	Leer RBR con la FIFO vacía no cambia el estado de la FIFO (el dato devuelto no se compara)	Scoreboard	cg_rx.cp_rd_empty	+UART_SW_LSR_FIRST_PCT, +UART_REG_NOISE_PCT
V-INT-06	F-INT-06	Sin interrupción de una fuente enmascarada en IER	Modelo de interrupciones	cg_int.cp_ier	+UART_IER_BIT_PCT
V-INT-01	F-INT-01	IIR = estado de línea ante error de paridad, trama, break u overrun con IER[2] = 1	Modelo de interrupciones	cg_int.cp_iir_id[RLS]	+UART_SER_PARITY_ERR_PCT, +UART_SER_FRAME_ERR_PCT, +UART_SER_BREAK_PCT, +UART_IER_BIT_PCT
V-INT-02	F-INT-02	IIR = dato disponible al alcanzar el nivel de disparo con IER[0] = 1	Modelo de interrupciones	cg_int.cp_iir_id[RDA] × cg_fifo_ctrl.cp_tl	+UART_FCR_TL_W, +UART_SW_DELAY_MIN
V-INT-03	F-INT-03	IIR[3:0] = 4’hC con datos en la FIFO y 4 tiempos de carácter sin actividad	Modelo de interrupciones	cg_int.cp_iir_id[TIMEOUT]	+UART_SER_RX_GAP_W, +UART_SW_PARTIAL_PCT, +UART_SW_DELAY_MAX
V-INT-04	F-INT-04	IIR = THRE con la FIFO de TX vacía e IER[1] = 1	Modelo de interrupciones	cg_int.cp_iir_id[THRE]	+UART_TX_BURSTS_MAX, +UART_TX_BLIND_PCT, +UART_IER_BIT_PCT

ID ítem	Característica	Qué se verifica	Mecanismo (scoreboard, SVA, RAL, cobertura)	Métrica de cierre (coverpoint o aserción)	Plusargs que lo dirigen
V-INT-05	F-INT-05	IIR = estado del módem ante un cambio en CTS, DSR, RI o DCD con IER[3] = 1	Modelo de interrupciones	cg_int.cp_iir_id[MSI] × cg_modem.cp_sig	+UART_MDM_CHANGES_MAX, +UART_MDM_SIG_W
V-INT-08	F-INT-08	IIR muestra la interrupción de mayor prioridad cuando hay varias pendientes	Modelo de interrupciones	cg_int.cp_multi, iir_id × multi	+UART_IER_BIT_PCT, +UART_SW_DELAY_MIN, +UART_SER_PARITY_ERR_PCT
V-FIFO-01	F-FIFO-01	El nivel de disparo (1, 4, 8, 14) decide cuándo se activa la interrupción de dato disponible	Modelo de interrupciones	cg_fifo_ctrl.cp_tl × cg_int.cp_iir_id[RDA]	+UART_FCR_TL_W
V-TX-04	F-TX-04	FIFO de TX vacía tras escribir FCR[2], sin cortar el carácter en curso	Scoreboard TX	cg_fifo_ctrl.cp_tx_clr × cp_traffic	+UART_FCR_CLR_PCT, +UART_REG_FIFO_CLR_PCT
V-RX-03	F-RX-03	FIFO de RX vacía tras escribir FCR[1] (LSR[0] = 0), sin cortar el carácter en curso	Scoreboard RX	cg_fifo_ctrl.cp_rx_clr × cp_traffic	+UART_FCR_CLR_PCT, +UART_REG_FIFO_CLR_PCT
V-LCR-01	F-LCR-01	Datos TX y RX correctos con cada longitud (5 a 8 bits)	Scoreboard TX y RX	cg_line_cfg.cp_wls × cp_dir	+UART_LCR_WLS_W
V-LCR-02	F-LCR-02	La parada dura 1, 1.5 o 2 bits según LCR[2] y la longitud; el receptor revisa solo el primer stop (AMB-06)	Scoreboard (los monitores de stx y srx miden la parada)	cg_line_cfg.cp_stb × cp_wls, cg_rx.cp_short_stop	+UART_LCR_STB_W, +UART_SER_SHORT_STOP_PCT
V-LCR-03	F-LCR-03	Bit de paridad correcto, impar y par	Scoreboard TX y RX	cg_line_cfg.cp_par × cp_wls	+UART_LCR_PAR_W, +UART_SER_PARITY_ERR_PCT
V-LCR-04	F-LCR-04	Bit de paridad fijo en 0 o en 1 con paridad stick	Scoreboard TX y RX	cg_line_cfg.cp_par (stick 0 y stick 1)	+UART_LCR_PAR_W
V-LCR-05	F-LCR-05	stx_pad_o en 0 con LCR[6] = 1	Aserción A-SER-04 y scoreboard TX	A-SER-04 (assert y cover); cg_tx.cp_break	+UART_TX_BREAK_PCT, +UART_TX_BREAK_MAX
V-TX-01	F-TX-01	La trama en stx: inicio en 0, datos con el bit menos significativo primero, paridad y parada en 1; ancho de bit = 16 × DL ciclos	Aserción A-SER-02 (ancho del bit de inicio) y scoreboard TX	A-SER-02 (assert y cover); cg_line_cfg.cp_dir	+UART_LCR_WLS_W, +UART_LCR_PAR_W
V-MDM-01	F-MDM-01	Los pines RTS y DTR siguen a MCR[1:0] con la polaridad de la especificación (AMB-03)	Scoreboard (monitor de módem) y aserción A-MDM-01	A-MDM-01 (assert y cover); cg_modem.cp_out	+UART_MCR_OUT_PCT
V-MDM-02	F-MDM-02	Out1 y Out2 guardados en MCR y visibles en MSR solo en loopback	Scoreboard	cg_modem.cp_out × cp_loop	+UART_MCR_OUT_PCT, +UART_MCR_LOOP_PCT
V-LB-01	F-LB-01	stx_pad_o = 1 y datos de THR recibidos por RBR en loopback	Aserción A-SER-03 y scoreboard	A-SER-03 (assert y cover); cg_line_cfg.cp_loop, cg_modem.cp_loop	+UART_MCR_LOOP_PCT
V-LB-02	F-LB-02	MSR[7:4] igual a los bits de MCR en loopback	Scoreboard y aserción A-MDM-02	A-MDM-02 (assert y cover); cg_modem: señal × flanco × loopback; salida × loopback	+UART_MCR_LOOP_PCT, +UART_MCR_OUT_PCT

ID ítem	Característica	Qué se verifica	Mecanismo (scoreboard, SVA, RAL, cobertura)	Métrica de cierre (coverpoint o aserción)	Plusargs que lo dirigen
V-ERR-01	F-ERR-01	LSR[0] sube con el primer dato y baja al vaciar la FIFO	Scoreboard (LSR)	cg_err.cp_dr	+UART_SER_RX_CHARS_MAX, +UART_SW_PARTIAL_PCT
V-ERR-02	F-ERR-02	Con la FIFO llena, un carácter nuevo activa LSR[1] y los datos de la FIFO no cambian	Scoreboard (LSR y datos RX)	cg_err.cp_oe × cp_tl	+UART_SW_DELAY_MIN, +UART_FCR_TL_W, +UART_SER_RX_GAP_W
V-ERR-03	F-ERR-03	Un carácter con paridad errónea activa LSR[2]	Scoreboard	cg_err.cp_pe, cg_rx.cp_err	+UART_SER_PARITY_ERR_PCT
V-ERR-04	F-ERR-04	Un carácter con la parada en 0 activa LSR[3]	Scoreboard	cg_err.cp_fe	+UART_SER_FRAME_ERR_PCT
V-ERR-05	F-ERR-05	Línea en 0 por un carácter completo activa LSR[4] y entra un carácter nulo a la FIFO	Scoreboard	cg_err.cp_bi, cg_rx.cp_err[BREAK]	+UART_SER_BREAK_PCT, +UART_SER_BREAK_EXTRA_MAX
V-ERR-06	F-ERR-06	LSR[5] sube cuando la FIFO de TX se vacía y baja al escribir THR	Scoreboard	cg_err.cp_thre	+UART_TX_BLIND_PCT, +UART_TX_GAP_MAX
V-ERR-07	F-ERR-07	LSR[6] sube solo con la FIFO y el registro de desplazamiento vacíos	Scoreboard	cg_err.cp_temt (cruce thre × temt)	+UART_TX_BLIND_PCT, +UART_TX_GAP_MAX
V-ERR-08	F-ERR-08	LSR[7] vale 1 mientras haya un error en la FIFO	Scoreboard	cg_err.cp_fifo_err	+UART_SER_PARITY_ERR_PCT, +UART_SER_FRAME_ERR_PCT
V-ERR-09	F-ERR-09	Al leer LSR se limpian los bits de error y baja la interrupción de estado de línea	Scoreboard y modelo de interrupciones	cg_err (oe, pe, fe, bi) × lectura; cg_int.cp_iir_id[RLS]	+UART_SW_LSR_FIRST_PCT
V-MDM-03	F-MDM-03	MSR[7:4] con la polaridad de la especificación (AMB-03)	Scoreboard	cg_modem: señal × flanco × loopback	+UART_MDM_SIG_W, +UART_MDM_CHANGES_MAX
V-MDM-04	F-MDM-04	Un cambio en CTS, DSR o DCD activa su delta	Scoreboard	cg_modem.cp_delta	+UART_MDM_CHANGES_MAX, +UART_MDM_MULTI_PCT
V-MDM-05	F-MDM-05	TERI se activa solo cuando RI cambia de bajo a alto (AMB-04)	Scoreboard	cg_modem.cp_teri	+UART_MDM_SIG_W, +UART_MDM_CHANGES_MAX
V-MDM-06	F-MDM-06	Leer MSR limpia los deltas	Scoreboard	cg_modem.cp_delta × lectura de MSR	+UART_MDM_GAP_MAX
V-DL-01	F-DL-01	DLL y DLM se escriben y se leen con DLAB = 1 y forman un valor de 16 bits	RAL (puerta delantera y trasera, regs.dl)	cg_dl.cp_dl	+UART_DL_MAX_FAST, +UART_DL_MAX
V-DL-02	F-DL-02	El ancho de bit en stx y el muestreo en srx es 16 × DL ciclos	Aserción A-SER-02 y scoreboard	A-SER-02 (assert y cover); cg_line_cfg.cp_dl_band	+UART_DL_MAX_FAST, +UART_DL_TAIL_W
V-DL-03	F-DL-03	Sin salida por stx con DL = 0; funcionamiento correcto con DL = 65535	Aserción A-SER-01 y scoreboard TX	A-SER-01 (assert y cover, DL = 0); cg_dl.cp_dl (0 y 65535)	+UART_DL_ZERO_PCT, +UART_DL_MAX

ID ítem	Característica	Qué se verifica	Mecanismo (scoreboard, SVA, RAL, cobertura)	Métrica de cierre (coverpoint o aserción)	Plusargs que lo dirigen
V-DL-04	F-DL-04	Divisor correcto con ambos órdenes de escritura (DLL al final inicia la cuenta)	RAL y scoreboard	cg_dl.cp_order	+UART_DL_LSB_FIRST_PCT
V-REG-03	F-REG-03	Campos de Debug 1 y Debug 2 correctos (AMB-08)	RAL y scoreboard	cg_reg_access.cp_addr (8 y 12)	+UART_REG_NOISE_PCT
V-WB-02	F-WB-02	Byte correcto en el carril que indica wb_sel_i	Scoreboard y adaptador del RAL	cg_reg_access.cp_sel × cp_rw	+UART_REG_NOISE_PCT
V-WB-03	F-WB-03	Con sel ilegal el UART no se cuelga, genera ack y aplica los efectos del RTL (AMB-05)	Aserción A-WB-05 y scoreboard	A-WB-05 (assert y cover); cg_reg_access.cp_sel[BAD]	+UART_WB_BAD_SEL_PCT
V-REG-02	F-REG-02	Valor de reset correcto en cada registro	RAL (uvm_reg_hw_reset_seq, puertas delantera y trasera)	cg_reset.cp_when; chequeo de reset	+UART_REG_HW_RESET_CHK
V-RST-01	F-RST-01	FIFOs vacías, stx en 1 e int_o en 0 tras el reset	Aserción A-RST-01, RAL y scoreboard	A-RST-01 (assert y cover); cg_reset.cp_when (reposo)	+UART_RST_PCT
V-RX-01	F-RX-01	Muestreo a la mitad del bit, pulsos cortos descartados y falso inicio detectado	Scoreboard RX (A-ENV-01 evita X en srx)	cg_rx.cp_err[GLITCH_REJ] y [GLITCH_START]	+UART_SER_GLITCH_PCT, +UART_SER_GLITCH_W_MAX
V-RX-04	F-RX-04	Dato correcto con desviación de baudios dentro de la tolerancia	Scoreboard RX	cg_rx.cp_err[BAUD_IN] y [BAUD_OUT]	+UART_SER_BAUD_IN_PCT, +UART_SER_BAUD_IN_MAX, +UART_SER_BAUD_OUT_PCT, +UART_SER_BAUD_OUT_MIN, +UART_SER_BAUD_OUT_MAX
V-TX-03	F-TX-03	Escrituras con la FIFO de TX llena se pierden sin alterar su contenido	Scoreboard TX	cg_tx.cp_level (lleno y perdido)	+UART_TX_LEN_W, +UART_TX_BLIND_PCT
V-REG-05	F-REG-05	Sin bloqueo ni chequeos rotos al reescribir LCR, DL, FCR o MCR con tráfico activo	Scoreboard (marca “no predecible” el carácter afectado)	cg_reconfig.cp_reg × cp_traffic	+UART_REG_RECONFIG_PCT, +UART_REG_FIFO_CLR_PCT

6. Estrategia de estímulo: prueba base

La prueba base se define en la tabla siguiente: qué es, qué hace en cada fase y cuándo pasa. El resto de la sección da el detalle del estímulo.

Aspecto	Definición
Nombre y clase	uart_base_test, extiende uvm_test y se registra en la factory, así que cualquier componente se puede sustituir con +uvm_set_type_override. Es la única prueba del proyecto.
Objetivo	Ejercitar todo lo que el UART debe cumplir (sección 4) con estímulo aleatorio restringido, y dejar cada caso alcanzable con los valores por defecto (sección 6.16).
build_phase	Crea uart_env_cfg (con uart_line_cfg y uart_ctrl_cfg) y los objetos de los agentes, scoreboard y cobertura. Llama a parse_plusargs(), el único lugar donde se leen plusargs. Toma las interfaces virtuales que publicó tb_top y las guarda en los objetos de los agentes. Publica todo con uvm_config_db (sección 9.3) y crea uart_env con la factory.
end_of_elaboration_phase	Imprime la topología del ambiente y la tabla de knobs con su valor y su origen (por defecto o plusarg), para poder reproducir la corrida.
run_phase	Levanta una objeción, arranca uart_main_vseq en el secuenciador virtual (sección 6.17) y baja la objeción cuando la secuencia termina. El drain time es de 2 Tc (tiempo de carácter, sección 6.4) del último formato, para que termine el carácter que esté en la línea.
Estímulo	Lo genera uart_main_vseq por episodios: configuración aleatoria, tráfico TX y RX con errores, cambios de módem, servicio del software por interrupciones o sondeo, y eventos raros (secciones 6.5 a 6.13).

Aspecto	Definición
Chequeo	Scoreboard con modelo de referencia (sección 10), aserciones SVA (sección 11) y RAL: revisión de valores de reset al inicio y del espejo en cada lectura (sección 12).
Límite de tiempo	Cada episodio tiene un límite (+UART_SEQ_EP_TIMEOUT, en Tc). Si se vence, la prueba da UVM_ERROR con el estado del modelo, para que un UART trabado no deje la simulación corriendo para siempre.
check_phase y report_phase	El scoreboard revisa que no queden datos esperados sin comparar. La prueba imprime PASA o FALLA y el conteo de errores conocidos del DUT (AMB-03 y AMB-04).
Criterio de paso	UVM_ERROR = 0, UVM_FATAL = 0, ninguna aserción fallida y scoreboard vacío al final. Las diferencias conocidas del DUT se reportan con la severidad de +UART_SCB_DUT_BUG_SEV y no hacen fallar la corrida.
Cómo se corre	./simv +UVM_TESTNAME=uart_base_test +ntb_random_seed=N -cm_name base_N, y para un caso de esquina se agrega -f tests/caso.args.
Salidas	Log con la tabla de knobs y el resumen del scoreboard, una línea [COV_CURVE] por episodio y la base de cobertura de la corrida para fusionar con urg.

La prueba arranca con la semilla, la secuencia principal repite 5 episodios, los agentes mueven los datos por los pines y el scoreboard y la cobertura comparan y miden.

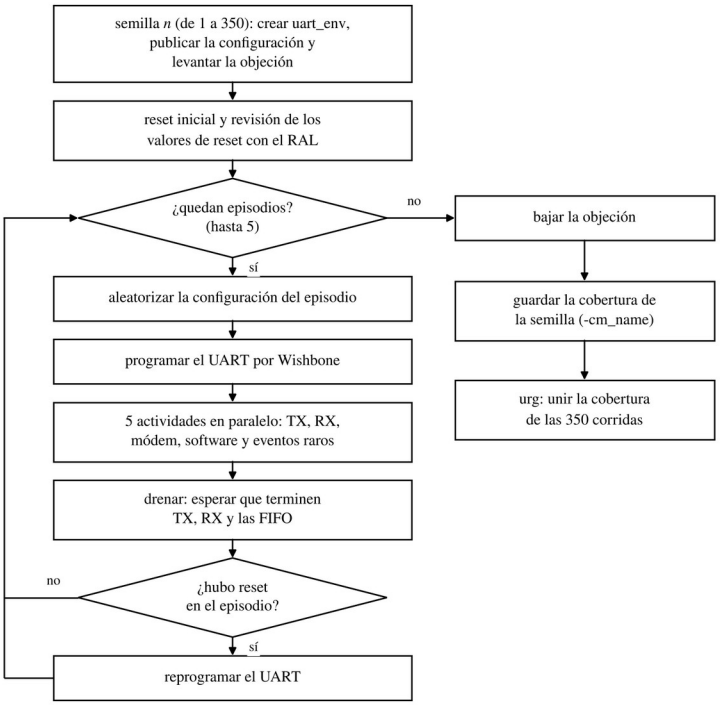


Figura 5. Flujo de una corrida del test base (una semilla).

6.1 Corrida y semilla

La regresión de línea base corre la prueba base con valores por defecto, 350 semillas y 5 episodios por semilla, es decir 1750 episodios en total. Antes de las 350, la primera verificación del ambiente usa 40 semillas (200 episodios) y exige que ninguna produzca errores falsos, es decir, errores que reporta el ambiente pero que no son defectos del UART (modelo, scoreboard, SVA o secuencias). El enunciado pide al menos 200 semillas o lo que se justifique por tiempo, así que 350 lo cumple, pero no admite un tiempo exagerado. El mínimo de 20 semillas sin errores falsos también se cumple, y la validación del ambiente usa 40 semillas para tener margen sobre ese mínimo.
Cada episodio es un sorteo independiente. Sea p la probabilidad de que un caso ocurra en un episodio (sale de las probabilidades por defecto de la sección 6.15). Con 5 episodios por semilla:

- Probabilidad de verlo al menos una vez en una semilla: $q = 1 - (1 - p)^5$.
- Probabilidad de verlo al menos una vez en N semillas: $1 - (1 - p)^{5\,N}$. Con N = 350 es $1 - (1 - p)^{1750}$.
- Semillas necesarias para tener 95 % de seguridad: $N = \ln(0.05) / \ln(1 - q)$.

Caso DL = 0, con p = 1 %. Primero $q = 1 - (0.99)^5 = 0.0490$, es decir 4.90 % por semilla. Después $N = \ln(0.05) / \ln(1 - 0.0490) = -2.996 / -0.0502 = 59.6$, que se redondea a 60 semillas. Con 350 semillas, la probabilidad es $1 - (0.99)^{1750}$, prácticamente 100 %.

Caso	p por episodio	q por semilla	Con 350 semillas	Semillas para 95 %
DL = 0	1 %	4.90 %	~100 %	60

Para el formato de trama, que tiene 80 combinaciones (longitud × parada × paridad × dirección) con p = 1/80 cada una, cubrirlas todas con 95 % de seguridad exige unos 587 episodios, es decir unas 118 semillas. Con 1750 episodios la seguridad es prácticamente total.
Los casos de 0.1 % o menos necesitan unas 600 semillas o más, y el valor exacto DL = 65535 es todavía más improbable. Subir el número de semillas para esos casos es caro, así que se cierran con casos de esquina. Asimismo, los escenarios con probabilidades muy cercanas a 0 % también se trabajan con plusargs.

6.2 Episodios

Cada corrida tiene 5 episodios fijos, uno tras otro. Un episodio es una configuración completa del UART con su tráfico. Los episodios son secuenciales porque hay un solo UART, no hay posibilidad de paralelismo. Entre un episodio y otro puede haber un reset. El número de episodios se fija con los plusargs +UART_SEQ_EP_MIN y +UART_SEQ_EP_MAX, que por defecto valen 5 y 5.
Se eligió esta estructura porque una sola corrida visita 5 configuraciones distintas, y porque prueba lo que pasa al reprogramar el UART y al hacer reset en medio de la corrida. La alternativa de un episodio por semilla es más simple y aísla mejor las fallas, pero necesita unas cinco veces más corridas para ver las mismas configuraciones. La contra de los episodios es que depurar una falla del episodio 5 obliga a reproducir la corrida desde el inicio con su semilla, por ello se descartó la opción de usar más de 5.

6.3 Pasos de un episodio

1. Se aleatoriza una configuración nueva (formato, divisor, FIFO, interrupciones y módem).
2. Se programa el UART por Wishbone: LCR con DLAB en 1, DLM y DLL en el orden elegido, LCR con DLAB en 0, y luego FCR, IER y MCR.
3. Se avisa a los agentes serie y de módem del formato nuevo.
4. Se ejecutan en paralelo cinco actividades: transmisión, recepción, módem, servicio del software y eventos raros. El episodio tiene un límite de tiempo (+UART_SEQ_EP_TIMEOUT, en Tc) para que un error del UART no deje la simulación esperando para siempre.
5. Se drena: se espera a que termine la transmisión (TEMT), se vacía la FIFO RX y se leen LSR, MSR e IIR para limpiar el estado. Con DL = 0 no se espera TEMT, porque el UART no transmite.
6. Si hubo reset, se espera a que termine y se reinician el espejo del RAL y el modelo del scoreboard. El episodio siguiente vuelve a programar el UART.

Antes del primer episodio la corrida hace un reset inicial y revisa los valores de reset con el RAL. Al final baja la objeción, cuando el último carácter en vuelo ya salió.

6.4 Tiempo de carácter

Un Tc es lo que dura un carácter completo: 16 × DL × (1 + bits de datos + paridad + bits de parada) ciclos. Con DL = 0 se usa el Tc de DL = 1.

6.5 Configuración y programación del UART

Al inicio de cada episodio se aleatorizan uart_line_cfg (LCR y DL) y uart_ctrl_cfg (FCR, IER y MCR) y se programan por Wishbone con la secuencia de DLAB.

6.6 Transmisión

De 0 a 6 ráfagas de escritura a THR por episodio, de 1 a 20 caracteres, con o sin revisar THRE, y break opcional en LCR[6]. El scoreboard de transmisión compara cada carácter observado en la línea contra los escritos en THR, y el estado de THRE y TEMT contra el modelo. Una aserción vigila la línea serie.

6.7 Recepción

El agente serie manda de 0 a 40 caracteres por episodio, con separación variable y con errores inyectados (paridad, trama, break, glitch y desviación de baudios). El scoreboard de recepción compara cada carácter leído en RBR contra el enviado, y los bits de LSR (PE, FE, BI, OE y DR) contra el modelo.

6.8 Módem

De 0 a 8 cambios por episodio en CTS, DSR, RI y DCD, y MCR aleatorio con loopback en el 20 % de los episodios.

6.9 Eventos raros pero que sí se pueden dar

Accesos a direcciones al azar, reescritura de LCR, DL, FCR o MCR con tráfico, limpieza de FIFO y reset en un momento aleatorio, además de pausas y retrasos en el bus Wishbone.

6.10 ¿Qué se aleatoriza?

Lo que cambia en cada episodio se agrupa en cinco familias: registros, tráfico, software que atiende, eventos raros y temporización del bus. Como se explicó antes, el objetivo es no necesitar demasiadas semillas ni prolongar el proceso.

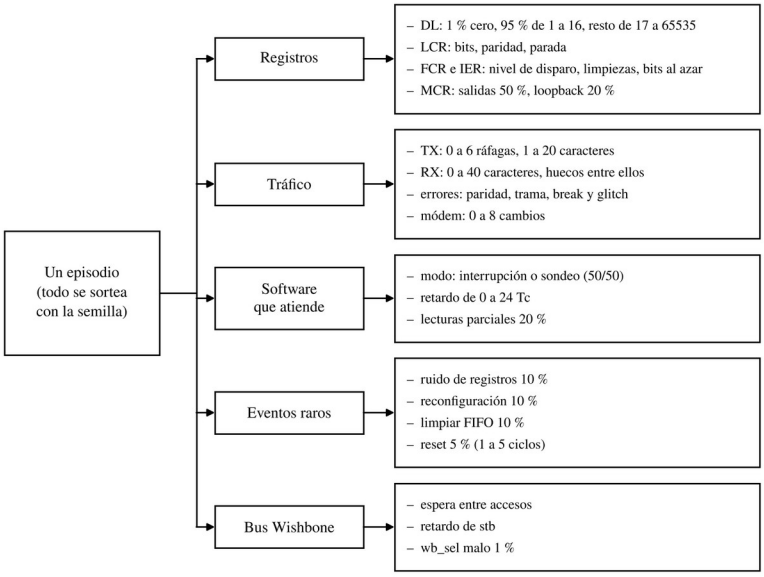


Figura 6. Mapa conceptual de lo que se aleatoriza en cada episodio.

6.11 Registros

En LCR, la longitud del carácter toma los cuatro valores (5 a 8 bits) con la misma probabilidad, los bits de parada toman 1 o 1.5/2 con 50 % cada uno, y la paridad elige uniformemente entre sus cinco modos: sin paridad, impar, par, stick 1 y stick 0. Cuando la paridad está deshabilitada, los bits de paridad par y stick también se aleatorizan, aunque no afecten la trama, porque así se mueven los bits del registro.

El divisor DL puede valer de 0 a 65535, pero se reparte de forma desigual por tiempo de simulación: 1 % en 0, 95 % entre 1 y 16 y 4 % en la cola (3 % entre 17 y 255, 0.8 % entre 256 y 4095, 0.2 % entre 4096 y 65535). DL = 0 es una configuración legal en la que el UART no transmite ni recibe, y se deja en 1 % para que se alcance sin plusargs. El orden de escritura es DLM primero el 90 % de las veces, que es lo que pide la especificación, y DLL primero el 10 %.

En FCR, el nivel de disparo (1, 4, 8 o 14 caracteres) se elige uniformemente, la limpieza de cada FIFO al configurar tiene 50 %, y los bits que el UART ignora también se aleatorizan. En IER, cada uno de los cuatro bits vale 1 con 50 %, así que se alcanzan las 16 combinaciones. En MCR, DTR, RTS, Out1 y Out2 valen 1 con 50 % cada uno, y loopback vale 1 con 20 %.

6.12 Tráfico

La transmisión se hace en ráfagas, de 0 a 6 por episodio. Cada ráfaga tiene entre 1 y 20 caracteres: 70 % entre 1 y 15, 15 % exactamente 16 y 15 % entre 17 y 20, para llenar y desbordar la FIFO de 16 posiciones. El 30 % de las ráfagas se escribe sin revisar THRE. Los datos son uniformes, pero el 20 % son valores de esquina (00, FF, 55 y AA). Con 5 % por episodio se activa el break de línea durante 1 a 4 Tc.

En la recepción, el agente serie manda de 0 a 40 caracteres por episodio (máximo 2 cuando DL es 256 o más, para que una semilla no tarde horas). La separación entre caracteres es 0 bits el 60 %, de 1 a 15 bits el 30 % y de 16 a 80 bits el 10 %; esta última supera los 4 Tc y provoca el timeout. Cada carácter puede llevar un error: paridad 5 %, trama 5 %, break 3 %, glitch 3 % y desviación de baudios 12 %. La desviación está dentro de la tolerancia (de 0.1 % a 2.5 %) o fuera (de 5 % a 10 %). Con 5 %, el agente manda un solo bit de parada aunque LCR pida 1.5 o 2.

Las señales de módem cambian de 0 a 8 veces por episodio, con la señal elegida uniformemente entre CTS, DSR, RI y DCD, una separación de 0 a 4 Tc y 10 % de cambios simultáneos de dos señales.

6.13 Software, eventos raros y bus

El software elige entre interrupciones y sondeo con 50 % cada uno. Su retraso de atención es de 0 a 1 Tc el 70 %, de 2 a 6 Tc el 22 % y de 7 a 24 Tc el 8 %. El 20 % de las veces lee menos datos de los disponibles y el 80 % lee LSR antes de RBR.

Con 10 % por iteración del software se hace un acceso a una dirección al azar (0 a 7, 8 o 12), y el 20 % de esos accesos es con DLAB en 1. Con 10 % por episodio se reescribe LCR, DL, FCR o MCR con tráfico activo, y con otro 10 % se limpia la FIFO

RX, la TX o ambas. Con 5 % hay un reset asíncrono en un momento al azar, de 1 a 5 ciclos de ancho. En el bus Wishbone, 60 % de los accesos van sin pausa y 40 % con 1 a 4 ciclos, el 20 % retrasa STB, y wb_sel_i es inválido en el 1 %.

6.14 Resumen de la prueba base

- **Idea central:** una sola prueba genérica, uart_base_test, que con valores aleatorios cubre cualquier configuración legal del UART y cualquier tipo de tráfico que soporte. Los casos de esquina salen de la misma prueba con plusargs que cambian las probabilidades.
- **Qué se verifica:** el formato de la trama (LCR) y el divisor (DL), el uso de las FIFO, los errores en la línea serie, las fuentes de interrupción y cómo las atiende el software, las señales de módem con el loopback, y el reset.
- **Ambiente:** uart_base_test crea uart_env, que contiene el secuenciador virtual, el RAL, los agentes Wishbone, serie y módem, el scoreboard y uart_cov. Las aserciones SVA (uart_wb_sva y uart_ser_sva) son módulos enlazados con bind a uart_top desde tb_top.
- **Corrida:** 350 semillas consecutivas (de 1 a 350), cada una reproducible con su semilla y su archivo .args, y 5 episodios fijos y secuenciales por semilla (1750 en total). Antes se valida el ambiente con 40 semillas sin errores falsos.
- **Episodio:** se aleatoriza una configuración nueva, se programa el UART por Wishbone, se ejecutan en paralelo transmisión, recepción, módem, servicio del software y eventos raros, y se drena. Si hubo reset, se reinician el espejo del RAL y el modelo del scoreboard, y el episodio siguiente reprograma el UART.
- **Qué se aleatoriza:** cinco familias: registros, tráfico, software que atiende, eventos raros y temporización del bus.
- **Cobertura de casos:** con 350 semillas los casos de 0.8 % por episodio o más se ven prácticamente seguro y el de 0.2 % con 97 %. Los de 0.1 % o menos se cierran con casos de esquina y plusargs.
- **Límites:** el divisor se concentra en 1 a 16, el retraso del software llega a 24 Tc, las ráfagas llegan a 20 caracteres, se deja un hueco de 2.5 % a 5 % en la desviación de baudios y el reset está activo por defecto con 5 %. Todos se amplían con plusargs.

6.15 Espacio de aleatorización

Variable aleatoria	Objeto de configuración	Rango legal	Distribución por defecto	Knobs
n_episodes	uart_env_cfg	≥ 1	Fijo en 5 (ep_min = ep_max = 5)	ep_min, ep_max
lcr.wls	uart_line_cfg	0 a 3 (5 a 8 bits)	Uniforme	wls_w[4]
lcr.stb	uart_line_cfg	0 a 1 (1 o 1.5/2 bits de parada)	50/50	stb_w[2]
par_mode (define lcr.pen, eps y sp)	uart_line_cfg	ninguna, impar, par, stick 1, stick 0	Uniforme entre los 5	par_w[5]
lcr.eps y lcr.sp con paridad deshabilitada	uart_line_cfg	0 a 1	Uniforme (no afectan la trama, pero mueven los bits del registro)	ninguno
dl	uart_line_cfg	0 a 65535	1 % en 0; 95 % en 1 a 16; 4 % en la cola: 3 % en 17 a 255, 0.8 % en 256 a 4095 y 0.2 % en 4096 a 65535	dl_zero_pct, dl_max_fast, dl_max, dl_tail_w[3] (por mil)
dl_order	uart_line_cfg	DLM primero o DLL primero	90 % DLM primero (lo que pide la especificación), 10 % DLL primero	dl_lsb_first_pct
fcr.tl	uart_ctrl_cfg	0 a 3 (1, 4, 8, 14)	Uniforme	tl_w[4]
fcr.rx_clr y fcr.tx_clr al configurar	uart_ctrl_cfg	0 a 1	50 % cada uno	fcr_clr_pct
fcr bits 0, 3 y 5:4	uart_ctrl_cfg	0 a 1	Uniforme (el DUT los ignora; sirven para toggle)	ninguno
ier[3:0]	uart_ctrl_cfg	0 a 15	Cada bit en 1 con 50 %	ier_bit_pct[4]
mcr.dtr, rts, out1 y out2	uart_ctrl_cfg	0 a 1	50 % cada uno	mcr_out_pct
mcr.loop	uart_ctrl_cfg	0 a 1	20 % en 1	mcr_loop_pct
sw_mode	uart_env_cfg	interrupciones o sondeo	50/50	sw_mode_w[2]
sw_delay (atención del software)	uart_env_cfg	≥ 0 Tc	70 % en 0 a 1 Tc; 22 % en 2 a 6 Tc; 8 % en 7 a 24 Tc	sw_delay_min, sw_delay_max, sw_delay_w[3]

Variable aleatoria	Objeto de configuración	Rango legal	Distribución por defecto	Knobs
sw_poll_period	uart_env_cfg	≥ 0 Tc	Uniforme en 0 a 4 Tc	sw_poll_max
sw_partial_read	uart_env_cfg	0 a 1	20 %: lee menos datos de los disponibles	sw_partial_pct
sw_lsr_first	uart_env_cfg	0 a 1	80 %: lee LSR antes de RBR (el resto lee RBR sin revisar, aunque la FIFO esté vacía)	sw_lsr_first_pct
tx_bursts por episodio	uart_env_cfg	≥ 0	Uniforme en 0 a 6	tx_bursts_max
tx_burst_len	uart_env_cfg	≥ 1	70 % en 1 a 15; 15 % exactamente 16; 15 % en 17 a 20 (desborda)	tx_len_max, tx_len_w[3]
tx_blind	uart_env_cfg	0 a 1	30 %: escribe la ráfaga sin revisar THRE	tx_blind_pct
tx_gap entre ráfagas	uart_env_cfg	≥ 0 Tc	Uniforme en 0 a 4 Tc	tx_gap_max
tx_data	uart_env_cfg	0 a 255	20 % esquinas (00, FF, 55, AA), 80 % uniforme	data_corner_pct
tx_break (LCR[6])	uart_env_cfg	0 a 1	5 % de los episodios, durante 1 a 4 Tc	tx_break_pct, tx_break_max
rx_chars por episodio	uart_ser_agent_cfg	≥ 0	Uniforme en 0 a 40 (máximo 2 si DL ≥ 256)	rx_chars_max, slow_dl, slow_chars_max
rx_gap entre caracteres	uart_ser_agent_cfg	≥ 0 bits	60 % en 0; 30 % en 1 a 15 bits; 10 % en 16 a 80 bits (más de 4 Tc, provoca timeout)	rx_gap_max, rx_gap_w[3]
rx_data	uart_ser_agent_cfg	0 a 255	Igual que tx_data	data_corner_pct
rx_err (por carácter)	uart_ser_agent_cfg	ninguno, paridad, trama, break, glitch, baudios	Paridad 5 %, trama 5 %, break 3 %, glitch 3 %, desviación de baudios 12 %; el resto sin error	parity_err_pct, frame_err_pct, break_pct, glitch_pct, baud_in_pct, baud_out_pct
break_len	uart_ser_agent_cfg	≥ 1 Tc	1 Tc más 0 a 2 Tc extra	break_extra_max
glitch_w (ancho del glitch)	uart_ser_agent_cfg	1 a 15 dieciseisavos de bit	Uniforme	glitch_w_max
baud_dev	uart_ser_agent_cfg	± 0.1 % a ± 10 %	Dentro de tolerancia (10 %): ± 0.1 % a 2.5 %. Fuera (2 %): ± 5 % a 10 %	baud_in_max, baud_out_min, baud_out_max (en décimas de %)
short_stop	uart_ser_agent_cfg	0 a 1	5 %: con LCR[2] = 1 el agente manda solo 1 bit de parada (AMB-06)	short_stop_pct
mdm_changes por episodio	modem_agent_cfg	≥ 0	Uniforme en 0 a 8	mdm_changes_max
mdm_sig	modem_agent_cfg	CTS, DSR, RI, DCD	Uniforme	mdm_sig_w[4]
mdm_gap	modem_agent_cfg	≥ 0 Tc	Uniforme en 0 a 4 Tc	mdm_gap_max
mdm_multi	modem_agent_cfg	0 a 1	10 %: cambian dos señales a la vez	mdm_multi_pct
reg_noise	uart_env_cfg	0 a 1 por iteración del software	10 %: acceso a una dirección al azar (0 a 7, 8 o 12), lectura o escritura	reg_noise_pct
reg_dlab	uart_env_cfg	0 a 1	20 % de esos accesos con DLAB = 1 (AMB-07)	reg_dlab_pct
reconfig	uart_env_cfg	0 a 1 por episodio	10 %: reescribe LCR, DL, FCR o MCR con tráfico activo	reconfig_pct
fifo_clr_mid	uart_env_cfg	0 a 1 por episodio	10 %: limpia la FIFO RX, la TX o ambas con tráfico activo	fifo_clr_mid_pct
wb_gap	wb_agent_cfg	≥ 0 ciclos	60 % en 0; 40 % en 1 a 4	wb_gap_max, wb_gap_zero_pct
wb_stb_delay	wb_agent_cfg	≥ 0 ciclos	80 % en 0; 20 % en 1 a 3 (cyc sube antes que stb)	wb_stb_delay_max, wb_stb_delay_pct
wb_bad_sel	wb_agent_cfg	0 a 1	1 %: wb_sel_i sin exactamente un bit activo (0000, dos bits o 1111) en registros de 8 bits (AMB-05)	wb_bad_sel_pct
rst_mid	uart_env_cfg	0 a 1 por episodio	5 %: reset asíncrono en un momento al azar del episodio	rst_pct
rst_width y rst_phase	uart_env_cfg	≥ 1 ciclo; 1 a 19 ns	Ancho uniforme en 1 a 5 ciclos; fase uniforme, no alineada al reloj	rst_width_max

Acotamientos y su justificación (sección 4.1 del enunciado).

Hay cinco acotamientos, todos con un plusarg que los amplía. El divisor se concentra en 1 a 16 porque con DL = 65535 un carácter 8N1 dura unos 10 millones de ciclos. El retraso del software llega a 24 Tc porque el peor caso de overrun es con nivel de disparo 1 (interrumpe con un dato y faltan 16 más para llenar la FIFO), y con 24 Tc alcanza. Las ráfagas llegan a 20 para desbordar la FIFO de 16 con al menos 4 escrituras perdidas. En los baudios se deja un hueco entre 2.5 % y 5 % de desviación, porque en esa zona el resultado depende de la fase exacta de muestreo y el scoreboard no puede predecirlo; los extremos

dentro y fuera de tolerancia sí son predecibles. El reset queda activo por defecto con 5 %, para que los casos de reset en medio del tráfico se alcancen sin plusargs, y se apaga con +UART_RST_PCT=0.

6.16 Argumento de completitud

Bin o grupo de bins	Por qué su probabilidad es mayor que cero con los valores por defecto	Plusarg que lo acelera
cg_line_cfg: wls × stb × par × dir (80 bins)	Las tres variables son independientes y todos sus pesos son mayores que 0 (1/4 × 1/2 × 1/5 por episodio). TX sale si hay ráfagas (6 de 7 episodios) y RX si rx_chars > 0	+UART_LCR_WLS_W, +UART_LCR_STB_W, +UART_LCR_PAR_W
cg_dl.cp_dl (0, 1, 2 a 16, 17 a 255, 256 a 4095, 4096 a 65534, 65535)	dl_zero_pct = 1 y cada banda de la cola tiene peso por mil mayor que 0; la última banda llega a 65535	+UART_DL_TAIL_W, +UART_DL_ZERO_PCT
cg_dl.cp_order[LSB_FIRST]	dl_lsb_first_pct = 10	+UART_DL_LSB_FIRST_PCT
cg_err.cp_oe × cp_tl	La FIFO RX se llena si sw_delay supera el tiempo de 16 − TL + 1 caracteres y llegan caracteres seguidos. sw_delay llega a 24 Tc con 8 % y rx_gap = 0 tiene 60 %	+UART_SW_DELAY_MIN, +UART_FCR_TL_W
cg_int.cp_iir_id[TIMEOUT]	Hace falta un dato en la FIFO y 4 Tc sin actividad. Pasa con rx_gap ≥ 16 bits (10 %) y TL > 1, o con lectura parcial (sw_partial_pct = 20)	+UART_SER_RX_GAP_W, +UART_SW_PARTIAL_PCT
cg_tx.cp_level[FULL] y [LOST]	tx_burst_len ≥ 17 con 15 % y tx_blind con 30 %	+UART_TX_LEN_W, +UART_TX_BLIND_PCT
cg_rx.cp_err × par × wls	Cada tipo de error tiene porcentaje mayor que 0 y se elige por carácter, independiente del formato. El error de paridad solo aplica con paridad habilitada (3 de 5 modos)	+UART_SER_PARITY_ERR_PCT, +UART_SER_FRAME_ERR_PCT
cg_rx.cp_err[BREAK] y cg_tx.cp_break	break_pct = 3 por carácter RX; tx_break_pct = 5 por episodio	+UART_SER_BREAK_PCT, +UART_TX_BREAK_PCT
cg_rx.cp_err[GLITCH_REJ] y [GLITCH_START]	glitch_w es uniforme en 1 a 15: el receptor revisa el bit de inicio a los 7/16 de bit, así que los anchos menores se descartan y los mayores generan un falso inicio	+UART_SER_GLITCH_PCT
cg_rx.cp_err[BAUD_IN] y [BAUD_OUT]	baud_in_pct = 10 y baud_out_pct = 2	+UART_SER_BAUD_OUT_PCT
cg_rx.cp_short_stop	Con LCR[2] = 1 (50 %), short_stop_pct = 5	+UART_SER_SHORT_STOP_PCT
cg_rx.cp_rd_empty (leer RBR con la FIFO vacía)	El 20 % de las veces el software lee RBR sin revisar LSR, y reg_noise lee la dirección 0 en cualquier momento	+UART_SW_LSR_FIRST_PCT, +UART_REG_NOISE_PCT
cg_int.cp_iir_id × cp_sw_mode	Cada bit de IER es independiente con 50 % y sw_mode es 50/50. Cada fuente tiene estímulo: errores RX, datos RX, ráfagas TX y cambios de módem	+UART_IER_BIT_PCT, +UART_SW_MODE_W
cg_int.cp_multi (dos o más fuentes pendientes)	Con IER = 1111 (1/16) y servicio lento (8 %) se acumulan errores, datos y THRE al mismo tiempo	+UART_SW_DELAY_MIN
cg_modem: sig × edge × loop	mdm_changes > 0 en 8 de 9 episodios, señales uniformes y loopback con 20 %. En loopback los cambios vienen de MCR, que también se aleatoriza	+UART_MDM_CHANGES_MAX, +UART_MCR_LOOP_PCT
cg_modem.cp_teri (RI sube y RI baja)	RI cambia en ambos sentidos con la misma probabilidad, lo que permite ver la diferencia de AMB-04	+UART_MDM_SIG_W=0,0,1,0
cg_reg_access: addr × rw × dlab	reg_noise_pct = 10 por iteración del software con todas las direcciones (0 a 7, 8, 12) uniformes y reg_dlab_pct = 20. Así se alcanzan las lecturas de LSR, MSR e IIR con DLAB = 1 (AMB-07). Además la configuración de cada episodio escribe todas las direcciones de control	+UART_REG_NOISE_PCT, +UART_REG_DLAB_PCT
cg_reg_access.cp_sel[BAD]	wb_bad_sel_pct = 1	+UART_WB_BAD_SEL_PCT
cg_reset.cp_when (reposo, TX, RX, ambos, durante un acceso)	rst_pct = 5 y el momento del reset es uniforme dentro del episodio	+UART_RST_PCT
cg_reconfig: reg × traffic	reconfig_pct = 10 y fifo_clr_mid_pct = 10, en un momento al azar del episodio	+UART_REG_RECONFIG_PCT, +UART_REG_FIFO_CLR_PCT

6.17 Secuencia virtual principal

```
uart_main_vseq::body()

  cfg = p_sequencer.cfg                    // knobs de la prueba
  esperar fin del reset inicial
  si hw_reset_chk: revisar valores de reset con el RAL

  n_episodes = valor entre EP_MIN y EP_MAX
  repetir n_episodes veces:
    line_cfg.randomize(); ctrl_cfg.randomize()   // con knobs
    programar_dut():
      LCR con DLAB = 1
      DLM y DLL en el orden de dl_order
      LCR final con DLAB = 0
      FCR (nivel y limpiezas), IER, MCR
    avisar a los agentes serie y de módem del formato nuevo

    fork
      tx_traffic()      // tx_bursts ráfagas a THR, con o sin revisar THRE; break opcional
```

```
      rx_traffic()      // el agente serie manda rx_chars caracteres con errores
      modem_traffic()   // mdm_changes cambios en cts, dsr, ri y dcd
      sw_service()      // interrupciones o sondeo, con sw_delay; incluye reg_noise
      eventos_raros()   // reconfig, fifo_clr_mid y reset en tiempos al azar
    join (con límite de tiempo por episodio: +UART_SEQ_EP_TIMEOUT)

    drenar():
      esperar TEMT (con límite de tiempo; con DL = 0 no se espera)
      vaciar la FIFO RX y leer LSR, MSR e IIR para limpiar
    si hubo reset: esperar su fin y reiniciar el espejo del RAL y el modelo

  fin: drop_objection; el drain time cubre el último carácter en vuelo
```

7. Tabla de plusargs

Plusarg	Tipo y rango	Valor por defecto	Componentes que lo usan	Objeto de configuración	Efecto
+UART_SEQ_EP_MIN	Entero ≥ 1	5	Secuencia virtual	uart_env_cfg	Mínimo de episodios por corrida
+UART_SEQ_EP_MAX	Entero ≥ EP_MIN	5	Secuencia virtual	uart_env_cfg	Máximo de episodios por corrida
+UART_SEQ_EP_TIMEOUT	Entero ≥ 1 (Tc)	2000	Secuencia virtual	uart_env_cfg	Tiempo máximo de un episodio; si se vence se reporta UVM_ERROR con el estado del modelo
+UART_LCR_WLS_W	4 enteros ≥ 0	1,1,1,1	Secuencia virtual, cobertura	uart_line_cfg	Pesos de 5, 6, 7 y 8 bits
+UART_LCR_STB_W	2 enteros ≥ 0	1,1	Secuencia virtual	uart_line_cfg	Pesos de 1 bit de parada y de 1.5/2
+UART_LCR_PAR_W	5 enteros ≥ 0	1,1,1,1,1	Secuencia virtual	uart_line_cfg	Pesos de ninguna, impar, par, stick 1 y stick 0
+UART_DL_ZERO_PCT	Entero 0 a 100	1	Secuencia virtual	uart_line_cfg	Porcentaje de episodios con DL = 0
+UART_DL_MAX_FAST	Entero 1 a 65535	16	Secuencia virtual	uart_line_cfg	Límite superior del rango rápido de DL
+UART_DL_MAX	Entero 1 a 65535	65535	Secuencia virtual	uart_line_cfg	Límite superior absoluto de DL
+UART_DL_TAIL_W	3 enteros, por mil	30,8,2	Secuencia virtual	uart_line_cfg	Peso de las bandas 17 a 255, 256 a 4095 y 4096 a DL_MAX
+UART_DL_LSB_FIRST_PCT	Entero 0 a 100	10	Secuencia de configuración	uart_line_cfg	Veces que se escribe DLL antes que DLM
+UART_FCR_TL_W	4 enteros ≥ 0	1,1,1,1	Secuencia virtual, cobertura	uart_ctrl_cfg	Pesos de los niveles de disparo 1, 4, 8 y 14
+UART_FCR_CLR_PCT	Entero 0 a 100	50	Secuencia virtual	uart_ctrl_cfg	Probabilidad de limpiar cada FIFO al configurar
+UART_IER_BIT_PCT	4 enteros 0 a 100	50,50,50,50	Secuencia virtual	uart_ctrl_cfg	Probabilidad de cada bit de IER (RDA, THRE, RLS, MS)
+UART_MCR_OUT_PCT	Entero 0 a 100	50	Secuencia virtual	uart_ctrl_cfg	Probabilidad de DTR, RTS, OUT1 y OUT2 en 1
+UART_MCR_LOOP_PCT	Entero 0 a 100	20	Secuencia virtual, scoreboard	uart_ctrl_cfg	Porcentaje de episodios en loopback
+UART_SW_MODE_W	2 enteros ≥ 0	1,1	Secuencia de servicio	uart_env_cfg	Pesos de interrupciones y sondeo
+UART_SW_DELAY_MIN	Entero ≥ 0 (Tc)	0	Secuencia de servicio	uart_env_cfg	Retraso mínimo de atención
+UART_SW_DELAY_MAX	Entero ≥ MIN (Tc)	24	Secuencia de servicio	uart_env_cfg	Retraso máximo de atención
+UART_SW_DELAY_W	3 enteros ≥ 0	70,22,8	Secuencia de servicio	uart_env_cfg	Pesos de retraso corto, medio y largo
+UART_SW_POLL_MAX	Entero ≥ 0 (Tc)	4	Secuencia de servicio	uart_env_cfg	Periodo máximo de sondeo
+UART_SW_PARTIAL_PCT	Entero 0 a 100	20	Secuencia de servicio	uart_env_cfg	Probabilidad de leer menos datos de los disponibles
+UART_SW_LSR_FIRST_PCT	Entero 0 a 100	80	Secuencia de servicio	uart_env_cfg	Probabilidad de leer LSR antes de RBR
+UART_TX_BURSTS_MAX	Entero ≥ 0	6	Secuencia TX	uart_env_cfg	Ráfagas máximas por episodio
+UART_TX_LEN_MAX	Entero ≥ 17	20	Secuencia TX	uart_env_cfg	Largo máximo de una ráfaga

Plusarg	Tipo y rango	Valor por defecto	Componentes que lo usan	Objeto de configuración	Efecto
+UART_TX_LEN_W	3 enteros ≥ 0	70,15,15	Secuencia TX	uart_env_cfg	Pesos de ráfaga corta, exactamente 16 y desborde
+UART_TX_BLIND_PCT	Entero 0 a 100	30	Secuencia TX	uart_env_cfg	Ráfagas escritas sin revisar THRE
+UART_TX_GAP_MAX	Entero ≥ 0 (Tc)	4	Secuencia TX	uart_env_cfg	Pausa máxima entre ráfagas
+UART_TX_BREAK_PCT	Entero 0 a 100	5	Secuencia TX	uart_env_cfg	Episodios con break en TX (LCR[6])
+UART_TX_BREAK_MAX	Entero ≥ 1 (Tc)	4	Secuencia TX	uart_env_cfg	Duración máxima del break TX
+UART_DATA_CORNER_PCT	Entero 0 a 100	20	Secuencias TX y RX	uart_env_cfg	Datos de esquina (00, FF, 55, AA)
+UART_SER_RX_CHARS_MAX	Entero ≥ 0	40	Secuencia RX	uart_ser_agent_cfg	Caracteres RX máximos por episodio
+UART_SER_SLOW_DL	Entero ≥ 1	256	Secuencia RX	uart_ser_agent_cfg	DL desde el cual se limita el tráfico
+UART_SER_SLOW_CHARS_MAX	Entero ≥ 1	2	Secuencia RX	uart_ser_agent_cfg	Caracteres máximos con DL lento
+UART_SER_RX_GAP_MAX	Entero ≥ 16 (bits)	80	Driver serie	uart_ser_agent_cfg	Pausa máxima entre caracteres RX
+UART_SER_RX_GAP_W	3 enteros ≥ 0	60,30,10	Driver serie	uart_ser_agent_cfg	Pesos de pausa 0, corta y larga
+UART_SER_PARITY_ERR_PCT	Entero 0 a 100	5	Driver serie, scoreboard	uart_ser_agent_cfg	Caracteres RX con paridad corrupta
+UART_SER_FRAME_ERR_PCT	Entero 0 a 100	5	Driver serie, scoreboard	uart_ser_agent_cfg	Caracteres RX con el bit de parada en 0
+UART_SER_BREAK_PCT	Entero 0 a 100	3	Driver serie, scoreboard	uart_ser_agent_cfg	Caracteres RX reemplazados por break
+UART_SER_BREAK_EXTRA_MAX	Entero ≥ 0 (Tc)	2	Driver serie	uart_ser_agent_cfg	Duración extra máxima del break
+UART_SER_GLITCH_PCT	Entero 0 a 100	3	Driver serie, scoreboard	uart_ser_agent_cfg	Pulsos en 0 aislados con la línea en reposo
+UART_SER_GLITCH_W_MAX	Entero 1 a 15 (1/16 de bit)	15	Driver serie	uart_ser_agent_cfg	Ancho máximo del glitch
+UART_SER_BAUD_IN_PCT	Entero 0 a 100	10	Driver serie	uart_ser_agent_cfg	Caracteres con desviación dentro de tolerancia
+UART_SER_BAUD_IN_MAX	Entero (0.1 %)	25	Driver serie	uart_ser_agent_cfg	Desviación máxima dentro de tolerancia
+UART_SER_BAUD_OUT_PCT	Entero 0 a 100	2	Driver serie, scoreboard	uart_ser_agent_cfg	Caracteres con desviación fuera de tolerancia
+UART_SER_BAUD_OUT_MIN	Entero (0.1 %)	50	Driver serie	uart_ser_agent_cfg	Desviación mínima fuera de tolerancia
+UART_SER_BAUD_OUT_MAX	Entero (0.1 %)	100	Driver serie	uart_ser_agent_cfg	Desviación máxima fuera de tolerancia
+UART_SER_SHORT_STOP_PCT	Entero 0 a 100	5	Driver serie	uart_ser_agent_cfg	Con 1.5/2 bits de parada, mandar solo 1
+UART_MDM_CHANGES_MAX	Entero ≥ 0	8	Secuencia de módem	modem_agent_cfg	Cambios máximos por episodio
+UART_MDM_SIG_W	4 enteros ≥ 0	1,1,1,1	Secuencia de módem	modem_agent_cfg	Pesos de CTS, DSR, RI y DCD
+UART_MDM_GAP_MAX	Entero ≥ 0 (Tc)	4	Driver de módem	modem_agent_cfg	Pausa máxima entre cambios
+UART_MDM_MULTI_PCT	Entero 0 a 100	10	Secuencia de módem	modem_agent_cfg	Cambios de dos señales a la vez
+UART_REG_NOISE_PCT	Entero 0 a 100	10	Secuencia de servicio	uart_env_cfg	Accesos extra a direcciones al azar
+UART_REG_DLAB_PCT	Entero 0 a 100	20	Secuencia de servicio	uart_env_cfg	De esos accesos, cuántos con DLAB = 1
+UART_REG_RECONFIG_PCT	Entero 0 a 100	10	Secuencia virtual	uart_env_cfg	Episodios con reconfiguración en medio del tráfico
+UART_REG_FIFO_CLR_PCT	Entero 0 a 100	10	Secuencia virtual	uart_env_cfg	Episodios con limpieza de FIFO en medio del tráfico
+UART_REG_HW_RESET_CHK	0 o 1	1	Secuencia virtual, RAL	uart_env_cfg	Correr la revisión de valores de reset al inicio

Plusarg	Tipo y rango	Valor por defecto	Componentes que lo usan	Objeto de configuración	Efecto
+UART_WB_GAP_MAX	Entero ≥ 0 (ciclos)	4	Driver Wishbone	wb_agent_cfg	Pausa máxima entre transacciones
+UART_WB_GAP_ZERO_PCT	Entero 0 a 100	60	Driver Wishbone	wb_agent_cfg	Transacciones seguidas sin pausa
+UART_WB_STB_DELAY_PCT	Entero 0 a 100	20	Driver Wishbone	wb_agent_cfg	Transacciones con stb retrasado respecto a cyc
+UART_WB_STB_DELAY_MAX	Entero ≥ 0 (ciclos)	3	Driver Wishbone	wb_agent_cfg	Retraso máximo de stb
+UART_WB_BAD_SEL_PCT	Entero 0 a 100	1	Driver Wishbone, predictor	wb_agent_cfg	Accesos con wb_sel_i sin exactamente un bit activo
+UART_RST_PCT	Entero 0 a 100	5	Secuencia virtual, scoreboard	uart_env_cfg	Episodios con reset asíncrono en medio
+UART_RST_WIDTH_MAX	Entero ≥ 1 (ciclos)	5	Secuencia virtual	uart_env_cfg	Ancho máximo del reset
+UART_SCB_EN	Lista de nombres	all	Scoreboard	uart_scb_cfg	Chequeos activos (tx, rx, lsr, iir, msr, int)
+UART_SCB_MAX_ERRORS	Entero ≥ 0	10	Scoreboard	uart_scb_cfg	Detener la corrida tras N errores (0 = nunca); los errores conocidos del DUT (AMB-03, AMB-04) no cuentan
+UART_SCB_DUT_BUG_SEV	warning o error	warning	Scoreboard	uart_scb_cfg	Severidad de las diferencias conocidas del DUT (AMB-03, AMB-04), también para la aserción A-MDM-01; cada una se cuenta y se lista al final de la corrida
+UART_COV_EN	0 o 1	1	Cobertura	uart_cov_cfg	Habilita la cobertura funcional
+UART_COV_CURVE_EVERY	Entero ≥ 0	1	Cobertura	uart_cov_cfg	Imprimir [COV_CURVE] cada N episodios (0 = no)

8. Casos de esquina planificados

ID	Escenario	Archivo .args	Plusargs	Bins objetivo	Semillas
CE-01	Overrun de la FIFO RX con TL 14 y 5 bits	rx_overrun_tl14_5bit.args	+UART_FCR_TL_W=0,0,0,1 +UART_LCR_WLS_W=1,0,0,0 +UART_SW_DELAY_MIN=6 +UART_SER_RX_GAP_W=1,0,0	cg_err.cp_oe × cp_tl[14]	20
CE-02	Overrun con TL 1	rx_overrun_tl1.args	+UART_FCR_TL_W=1,0,0,0 +UART_SW_DELAY_MIN=18 +UART_SER_RX_GAP_W=1,0,0	cg_err.cp_oe × cp_tl[1]	20
CE-03	Timeout con cada nivel de disparo	rx_timeout.args	+UART_SER_RX_GAP_W=0,0,1 +UART_SW_PARTIAL_PCT=60 +UART_IER_BIT_PCT=100,50,50,50	cg_int.cp_iir_id[TIMEOUT] × cp_tl	20
CE-04	Desborde de la FIFO TX	tx_overflow.args	+UART_TX_LEN_W=0,1,4 +UART_TX_BLIND_PCT=100	cg_tx.cp_level[FULL] y [LOST]	10
CE-05	Errores de paridad y trama en todos los formatos	rx_errors.args	+UART_SER_PARITY_ERR_PCT=30 +UART_SER_FRAME_ERR_PCT=30 +UART_LCR_PAR_W=0,1,1,1,1	cg_rx.cp_err × cp_par × cp_wls	30
CE-06	Break en RX y TX	break.args	+UART_SER_BREAK_PCT=40 +UART_SER_BREAK_EXTRA_MAX=4 +UART_TX_BREAK_PCT=80	cg_rx.cp_err[BREAK], cg_tx.cp_break, cg_err.cp_bi	20
CE-07	Glitches en la línea RX	rx_glitch.args	+UART_SER_GLITCH_PCT=50	cg_rx.cp_err[GLITCH_REJ] y [GLITCH_START]	10
CE-08	Desviación de baudios dentro y fuera de tolerancia	baud_dev.args	+UART_SER_BAUD_OUT_PCT=50 +UART_SER_BAUD_IN_PCT=40	cg_rx.cp_err[BAUD_IN] y [BAUD_OUT]	10
CE-09	Cola de DL (velocidades lentas)	dl_tail.args	+UART_DL_ZERO_PCT=0 +UART_DL_TAIL_W=600,300,100 +UART_SEQ_EP_MIN=3 +UART_SEQ_EP_MAX=3 +UART_SER_SLOW_CHARS_MAX=2	cg_dl.cp_dl[17:65535]	5
CE-10	DL = 0 y escritura del divisor con DLL primero	dl_zero_order.args	+UART_DL_ZERO_PCT=40 +UART_DL_LSB_FIRST_PCT=50	cg_dl.cp_dl[ZERO], cp_order	10

ID	Escenario	Archivo .args	Plusargs	Bins objetivo	Semillas
CE-11	Loopback con todos los formatos	loopback.args	+UART_MCR_LOOP_PCT=100	cg_line_cfg.cp_loop × cp_dir, cg_modem × cp_loop	20
CE-12	Muchos cambios de módem	modem_storm.args	+UART_MDM_CHANGES_MAX=40 +UART_MDM_MULTI_PCT=40 +UART_IER_BIT_PCT=50,50,50,100	cg_modem completo, cp_teri	10
CE-13	Prioridad de interrupciones con varias fuentes a la vez	int_priority.args	+UART_IER_BIT_PCT=100,100,100,100 +UART_SW_MODE_W=1,0 +UART_SW_DELAY_W=0,1,1 +UART_SER_PARITY_ERR_PCT=20	cg_int.cp_multi, cp_iir_id × cp_sw_mode	20
CE-14	Reset asíncrono frecuente	reset_storm.args	+UART_RST_PCT=60	cg_reset.cp_when × cp_width	20
CE-15	Ruido de registros, DLAB, scratch y Debug (AMB-01, AMB-07)	reg_noise.args	+UART_REG_NOISE_PCT=60 +UART_REG_DLAB_PCT=50 +UART_REG_RECONFIG_PCT=50 +UART_REG_FIFO_CLR_PCT=50	cg_reg_access, cg_reconfig, cg_rx.cp_rd_empty	20
CE-16	wb_sel_i inválido en el bus de 32 bits (AMB-05)	wb_bad_sel.args	+UART_WB_BAD_SEL_PCT=30	cg_reg_access.cp_sel[BAD] × cp_rw	10
CE-17	1.5 y 2 bits de parada con el agente mandando solo 1 (AMB-06)	stop_bits.args	+UART_LCR_STB_W=0,1 +UART_SER_SHORT_STOP_PCT=50	cg_line_cfg.cp_stb × cp_wls, cg_rx.cp_short_stop	10

9. Arquitectura del ambiente

La prueba, uart_base_test, crea el ambiente uart_env y le publica la configuración. Dentro de uart_env están el estímulo (el secuenciador virtual y el modelo de registros), los agentes y el chequeo con la cobertura.

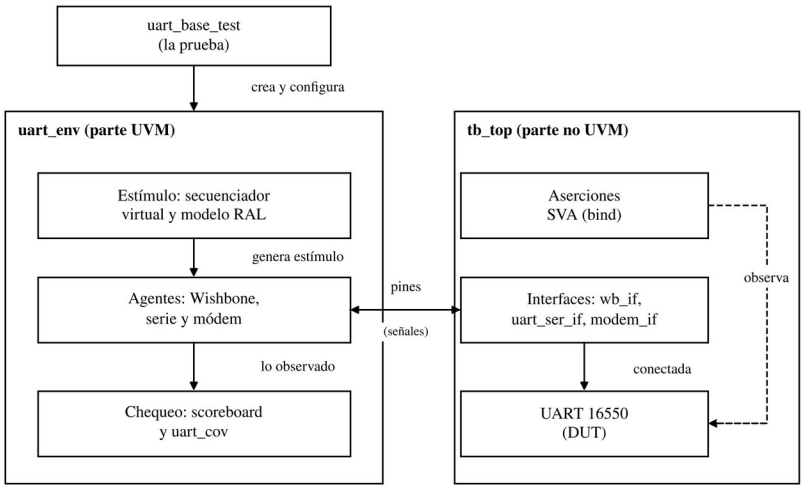


Figura 7. Ambiente de verificación UART.

Cada rama existe porque la prueba base la necesita:

- El secuenciador virtual lanza la secuencia principal y coordina a los tres agentes.
- El RAL (modelo, adaptador y predictor) permite leer y escribir los registros por la puerta delantera (Wishbone) y por la trasera (hdl_path a tb_top.dut.regs), y mantiene un espejo de los valores esperados.
- El agente Wishbone hace de CPU. Su driver genera los accesos a registros, su monitor los observa y uart_int_monitor vigila la salida int_o.
- El agente serie hace de dispositivo remoto: manda tramas por la línea de recepción, con errores cuando se le pide, y observa lo que el UART transmite.
- El agente de módem mueve las entradas CTS, DSR, RI y DCD y observa las salidas RTS y DTR.
- El scoreboard compara lo observado con un modelo de referencia (transmisión y recepción, y el estado de LSR, IIR, MSR e int_o).
- uart_cov recoge la cobertura funcional.

9.1 Diagrama

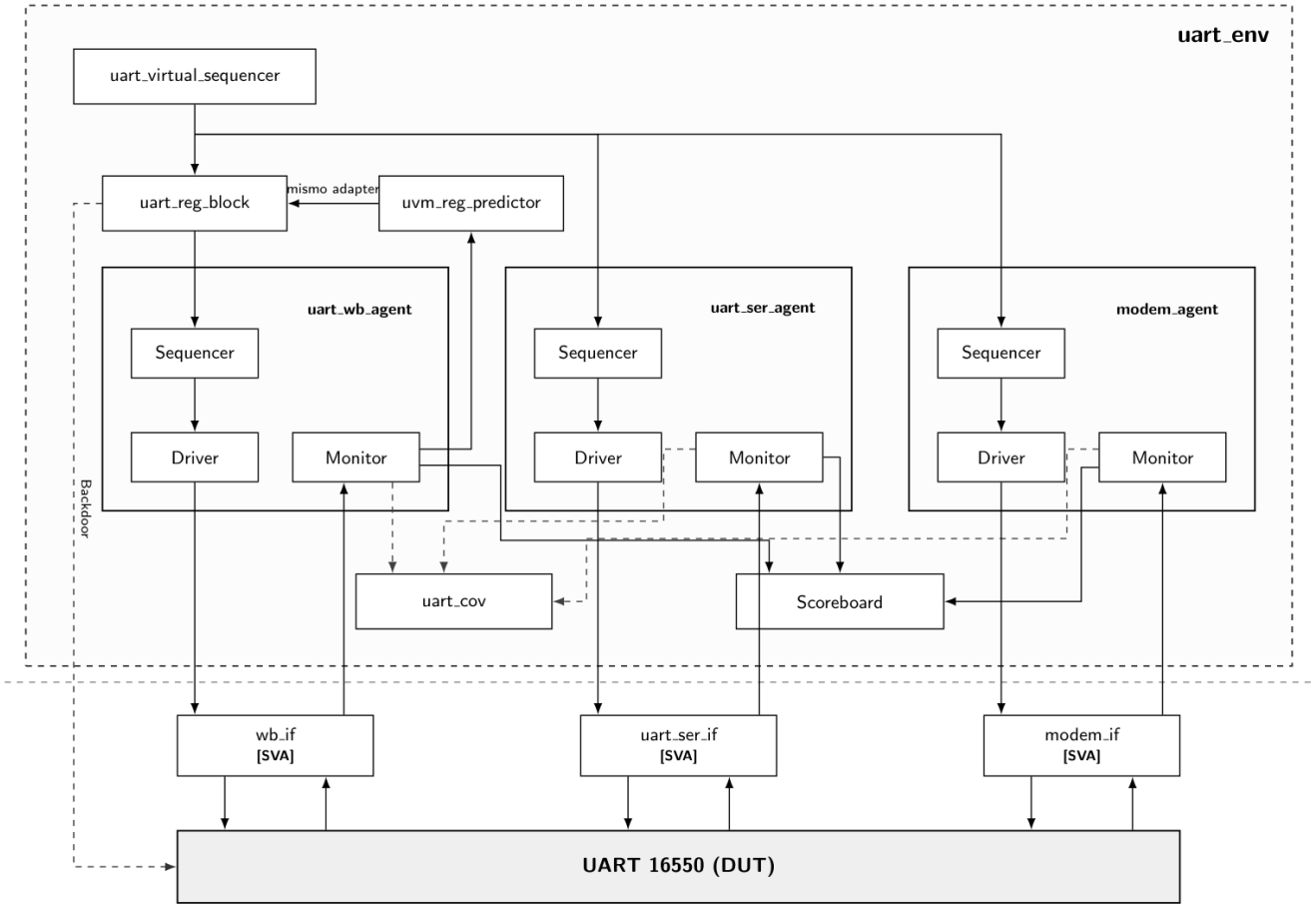


Figura 8. Arquitectura del ambiente.

9.2 Componentes

Componente	Clase UVM base	Responsabilidad	Puertos TLM
uart_base_test	uvm_test	Crea los objetos de configuración, llama a parse_plusargs(), publica todo en uvm_config_db y arranca la secuencia virtual principal	N/A
uart_env (instancia env)	uvm_env	Crea y conecta agentes, RAL, predictor, scoreboard y cobertura	N/A (hace las conexiones)
uart_virtual_sequencer	uvm_sequencer	Tiene las referencias a los secuenciadores Wishbone, serie y de módem, al RAL y a uart_env_cfg; ahí corre uart_main_vseq	N/A
uart_wb_agent (wb_agt)	uvm_agent	Driver maestro Wishbone con retardos y manejo de wb_sel_i, monitor del bus, secuenciador y el monitor de int_o (uart_int_monitor, que va dentro del agente)	ap (uvm_analysis_port) con cada transacción completada
uart_int_monitor	uvm_monitor	Observa int_o (wb_if.irq) y publica cada flanco para el scoreboard y la cobertura	ap (uvm_analysis_port)
uart_ser_agent (ser_agt)	uvm_agent	Driver hacia srx con inyección de errores; monitor de stx (caracteres TX) y monitor de srx (caracteres RX y error medido en la línea)	tx_ap y rx_ap (uvm_analysis_port)
modem_agent (mdm_agt)	uvm_agent	Driver de cts, dsr, ri y dcd; monitor de las cuatro entradas y de rts y dtr	ap (uvm_analysis_port)
uart_reg_block (regm)	uvm_reg_block	Modelo de registros con los mapas para DLAB = 0 y DLAB = 1 y rutas de puerta trasera a tb_top.dut.regs	N/A (frontdoor por adaptador, backdoor por hdl_path)
uart_reg_adapter	uvm_reg_adapter	reg2bus y bus2reg: convierte un acceso del RAL en una transacción Wishbone con el sel y el carril correctos	N/A

Componente	Clase UVM base	Responsabilidad	Puertos TLM
uart_reg_predictor	uvm_reg_predictor #(wb_item)	Actualiza el espejo del RAL con lo que ve el monitor Wishbone; elige el mapa según LCR[7] del espejo	bus_in (uvm_analysis_imp)
uart_scoreboard (scb)	uvm_scoreboard	Contiene el modelo de referencia (FIFOs, LSR, IIR, MSR, int_o) y compara contra lo observado	Un uvm_analysis_imp por fuente: wb, tx, rx, mdm e int (con uvm_analysis_imp_decl); ap de eventos del modelo hacia la cobertura
uart_cov (cov)	uvm_component	Covergroups de la sección 13, muestreados desde monitores y eventos del modelo	Un uvm_analysis_imp por fuente
uart_wb_sva y uart_ser_sva	module (bind a uart_top)	Aserciones de la sección 11	N/A

9.3 Flujo de configuración

Objeto de configuración	Ruta en uvm_config_db	Lo publica	Lo obtienen
Interfaces virtuales (wb_if, uart_ser_if, modem_if)	uvm_test_top* con los nombres wb_vif, ser_vif y mdm_vif	tb_top, antes de run_test()	uart_base_test, que las guarda en el campo vif de cada objeto de agente
uart_env_cfg (con referencias a uart_line_cfg y uart_ctrl_cfg)	uvm_test_top.env*	uart_base_test, después de parse_plusargs()	uart_env, secuenciador virtual (y por p_sequencer la secuencia virtual y sus secuencias), scoreboard, modelo de referencia y cobertura
wb_agent_cfg	uvm_test_top.env.wb_agt*	uart_base_test	Agente, driver, monitor y secuencias Wishbone (por p_sequencer), y el monitor de interrupción
uart_ser_agent_cfg	uvm_test_top.env.ser_agt*	uart_base_test	Agente serie, driver, monitores de stx y srx y secuencias RX
modem_agent_cfg	uvm_test_top.env.mdm_agt*	uart_base_test	Agente, driver, monitor y secuencias de módem
uart_scb_cfg	uvm_test_top.env.scb*	uart_base_test	Scoreboard y modelo de referencia
uart_cov_cfg	uvm_test_top.env.cov*	uart_base_test	Colector de cobertura uart_cov

10. Modelo de referencia y scoreboard

Qué se predice	Fuente de la predicción	Momento de la comparación	Tolerancia
Datos TX	Escrituras a THR vistas por el monitor Wishbone (las que llegan con la FIFO TX llena se descartan)	Cuando el monitor de stx reconstruye el carácter	Ninguna en el dato. Los caracteres afectados por reconfiguración, limpieza de FIFO o reset se marcan como no predecibles
Datos RX	Caracteres que el monitor de srx ve en la línea, con el error medido	Cuando el software lee RBR (monitor Wishbone)	Ninguna. Los caracteres con baudios fuera de tolerancia, falso inicio por glitch, o cruzados con reconfiguración o reset se marcan como no predecibles: no se compara el dato pero sí que la FIFO no pierda la cuenta
LSR	Modelo: nivel de las FIFOs, errores medidos por el monitor de srx y estado del transmisor	En cada lectura de LSR (valor leído contra el modelo)	Bits de error: ninguna. DR, THRE y TEMT: ventana de 2 ciclos alrededor del cambio, porque el RTL registra las banderas y THRE tiene el retardo de block_cnt
IIR	Fuente pendiente de mayor prioridad según el modelo y IER	En cada lectura de IIR	Ventana de 2 ciclos cuando cambia una fuente
MSR	Entradas que ve el monitor de módem, MCR (en loopback) y las decisiones AMB-03 y AMB-04	En cada lectura de MSR	2 ciclos de sincronización de las entradas. Las diferencias de AMB-03 y AMB-04 se reportan como error conocido del DUT, con la severidad de +UART_SCB_DUT_BUG_SEV
int_o	Fuentes pendientes habilitadas en IER según el modelo	En cada flanco que ve el monitor de interrupción y en cada lectura de IIR	1 a 2 ciclos (int_o e IIR se registran con términos de limpieza distintos; igual que A-INT-01)
rts_pad_o y dtr_pad_o	Escrituras a MCR con la polaridad de AMB-03	En cada cambio que ve el monitor de módem y después de cada escritura a MCR	1 ciclo; la diferencia de AMB-03 se reporta como error conocido del DUT
LCR, IER, DL y scratch	Espejo del RAL actualizado por el predictor	En cada lectura frontdoor (mirror con check) y por puerta trasera	Ninguna

11. Aserciones

Todas las aserciones están en módulos enlazados con bind a uart_top, como en el ejemplo uart_wb_sva del kit. Cada propiedad tiene su cover property, como pide el enunciado.

ID	Interfaz	Propiedad (en palabras)	Tipo (assert, assume, cover)	Ítem del plan
A-WB-01	Wishbone	adr, dat_i, we y sel estables mientras cyc y stb están activos y no hay ack	assert y cover	V-WB-01
A-WB-02	Wishbone	Con cyc y stb activos, ack llega en 4 ciclos o menos	assert y cover (ack a los 2 ciclos)	V-WB-01
A-WB-03	Wishbone	ack dura un solo ciclo y solo aparece con cyc y stb activos	assert y cover	V-WB-01
A-WB-04	Wishbone	cyc, stb, we, adr y sel sin X ni Z fuera de reset; ack y dat_o sin X cuando hay ack	assert y cover (transferencia completa sin X)	V-WB-01
A-WB-05	Wishbone	Un acceso con wb_sel_i sin exactamente un bit activo igual recibe ack (AMB-05)	assert y cover	V-WB-03
A-SER-01	Serie	stx en 1 mientras no haya carácter en curso, con LCR[6] = 0 y sin loopback. Con DL = 0 desde antes de empezar el carácter, stx sigue en 1 aunque haya datos en la FIFO TX (no aplica si DL pasa a 0 con un carácter a medias)	assert y cover (DL = 0 con datos en la FIFO TX)	V-DL-03
A-SER-02	Serie	El bit de inicio en stx dura 16 × DL ciclos	assert y cover (DL = 1 y DL > 16)	V-TX-01, V-DL-02
A-SER-03	Serie	En loopback stx se mantiene en 1	assert y cover	V-LB-01
A-SER-04	Serie	Con LCR[6] = 1, stx está en 0	assert y cover (break a mitad de carácter)	V-LCR-05
A-RST-01	Sistema	Después del reset, stx = 1 e int_o = 0	assert y cover (reset con tráfico activo)	V-RST-01, V-RST-02
A-INT-01	Interrupción	int_o = 1 si y solo si IIR[0] = 0 (regs.iir), con una ventana de 1 a 2 ciclos (int_o e iir se registran con términos de limpieza distintos)	assert y cover (sube y baja)	V-INT-07
A-MDM-01	Módem	rts_pad_o y dtr_pad_o son el complemento de MCR[1] y MCR[0] (AMB-03). Con el RTL actual falla y se reporta como error conocido del DUT, con la severidad de +UART_SCB_DUT_BUG_SEV (la acción de la aserción usa el mismo reporte que el scoreboard)	assert y cover	V-MDM-01
A-MDM-02	Módem	En loopback MSR[7:4] = OUT2, OUT1, DTR, RTS de MCR	assert y cover	V-LB-02
A-ENV-01	Serie y módem	srx y las entradas de módem nunca están en X o Z fuera de reset (restricción del ambiente)	assume y cover	V-RX-01

12. RAL

12.1 Registros y campos

Registro	Dir.	DLAB	Campo	Bits	Acceso	Reset	Ruta de puerta trasera
Receiver Buffer (RBR)	0	0	RBR (Data)	7:0	R	8'h00	Sin ruta: es la salida de la FIFO RX (lo revisa el scoreboard)
Transmitter Holding (THR)	0	0	Transmit FIFO input	7:0	W	8'h00	Sin ruta: es la entrada de la FIFO TX (lo revisa el scoreboard)
Interrupt Enable (IER)	1	0	Received Data available interrupt	0	RW	8'h00	uart_top.regs.ier
			Transmitter Holding Register empty interrupt	1			
			Receiver Line Status Interrupt	2			
			Modem Status Interrupt	3			
			Reserved (se leen en 0)	7:4	RO		
Interrupt Identification (IIR)	2	X	Interrupt pending	0	RO	8'hC1	uart_top.regs.iir
			Interrupt ID / Priority Code	3:1			
			Reserved / logic 0	5:4			
			Compatibility reason / logic 1	7:6			
FIFO Control (FCR)	2	X	Ignored	0	W	8'hC0 (TL = 14)	uart_top.regs.fcr (solo bits 7:6)
			Clear Receiver FIFO	1			
			Clear Transmitter FIFO	2			
			Ignored	5:3			

Registro	Dir.	DLAB	Campo	Bits	Acceso	Reset	Ruta de puerta trasera
			Receiver FIFO Interrupt Trigger Level	7:6			
Line Control (LCR)	3	X	Select number of bits in each character	1:0	RW	8'h03	uart_top.regs.lcr
			Specify the number of generated stop bits	2			
			Parity Enable	3			
			Even Parity select	4			
			Stick Parity bit	5			
			Break Control bit	6			
			Divisor Latch Access bit	7			
Modem Control (MCR)	4	X	DTR Signal Control	0	WO	8'h00	uart_top.regs.mcr
			RTS Signal Control	1			
			Out 1	2			
			Out 2	3			
			Loopback mode	4			
			Ignored	7:5			
Line Status (LSR)	5	X	Data Ready indicator	0	RO	8'h60	uart_top.regs.lsr
			Overrun Error indicator	1	RC		
			Parity Error indicator	2	RC		
			Framing Error indicator	3	RC		
			Break Interrupt indicator	4	RC		
			Transmit FIFO is empty	5	RO		
			Transmitter Empty indicator	6	RO		
			FIFO Error indicator	7	RO		
Modem Status (MSR)	6	X	Delta Clear to Send indicator	0	RC	8'h00	uart_top.regs.msr
			Delta Data Set Ready indicator	1	RC		
			Trailing Edge of Ring Indicator Detector	2	RC		
			Delta Data Carrier Detect indicator	3	RC		
			Complement of the CTS input or equals to RTS in loopback mode	4	RO		
			Complement of the DSR input or equals to DTR in loopback mode	5			
			Complement of the RI input or equals to Out1 in loopback mode	6			
			Complement of the DCD input or equals to Out2 in loopback mode	7			
Divisor Latch byte 1 (LSB)	0	1	DLL	7:0	RW	8'h00	uart_top.regs.dl[7:0]
Divisor Latch byte 2 (MSB)	1	1	DLM	7:0	RW	8'h00	uart_top.regs.dl[15:8]
Debug 1	8	X	Line Status Register value	7:0	R	32'h00031060 (bits 31:28 no se comparan)	Sin ruta: es combinacional (uart_debug_if); se compara con los registros que copia
			Interrupt Enable Register value	11:8			
			Interrupt Identifier Register value	15:12			
			Line Control Register value	23:16			
			Modem Status Register value	31:24			
Debug 2	12	X	Transmitter FSM state	2:0	R	32'h00C00000	Sin ruta: es combinacional (uart_debug_if); se compara con los registros que copia
			Number of characters in Transmitter FIFO	7:3			
			Receiver FSM state	11:8			
			Number of characters in Receiver FIFO	16:12			
			Modem Control Register value	21:17			
			FIFO Control Register value (bits 7:6)	23:22			
			Reserved	31:24			
Scratch (SR)	7	X	Scratch	7:0	RW	8'h00	uart_top.regs.scratch

12.2 Decisiones de modelado

Tema	Decisión	Justificación
Registros con dirección compartida (DLAB)	Dos mapas en el reg_block con el mismo adaptador y el mismo secuenciador Wishbone: dlab0_map (RBR R y THR W en 0, IER en 1) y dlab1_map (DLL en 0, DLM en 1). El resto de registros está en los dos mapas. Antes de usar dlab1_map la secuencia pone LCR[7] = 1 y al terminar lo regresa a 0.	UVM deja que dos registros compartan dirección en un mapa solo si uno es R y el otro W (RBR y THR). DLL y DLM son RW, así que necesitan su propio mapa. Es lo más parecido a como el hardware multiplexa por DLAB.
Predictor y adaptador: conexión y política de predicción	Predicción explícita: uart_reg_predictor (extiende uvm_reg_predictor #(wb_item)) conectado al ap del monitor Wishbone, con el mismo adaptador. Auto predicción apagada en los dos mapas (set_auto_predict(0)) y regm.reset() después de construir. Como una dirección puede ser de dos registros, el predictor mira LCR[7] en el espejo para decidir en qué mapa predecir.	El espejo solo cambia con lo que realmente pasó por el bus con ack, incluyendo los accesos que no salen del RAL (ruido de registros, sel inválido). Sin mirar DLAB, el predictor actualizaría IER cuando en realidad se escribió DLM.
Efectos laterales (lectura que limpia, FIFOs)	LSR bits 1 a 4 y MSR bits 3:0: acceso RC. LSR[7] es RO: su parte de overrun se limpia al leer, pero el error de FIFO sigue mientras el carácter esté en la FIFO. Lectura de IIR: callback que limpia THRE pendiente en el espejo. RBR y THR: volátiles, sin chequeo de espejo; el flujo de datos lo revisa el scoreboard. MCR: WO; su valor se revisa por puerta trasera (AMB-02).	Las lecturas por frontdoor sacan datos de la FIFO y limpian banderas. Si el RAL intentara compararlas como registros normales daría errores falsos.
Secuencias integradas que se correrán y exclusiones	uvm_reg_hw_reset_seq (frontdoor y backdoor), uvm_reg_bit_bash_seq y uvm_reg_access_seq. RBR y THR: NO_REG_TESTS (son FIFOs). MCR: NO_REG_BIT_BASH_TEST y NO_REG_ACCESS_TEST, porque se lee 0x00 (AMB-02); se revisa por puerta trasera. LSR, MSR, IIR y FCR: NO_REG_BIT_BASH_TEST y NO_REG_ACCESS_TEST (estado volátil o solo escritura). Debug 1 y Debug 2: NO_REG_BIT_BASH_TEST (solo lectura, copian otros registros). MSR en hw_reset: se compara solo [3:0], porque [7:4] sigue a las entradas de módem desde el primer ciclo. Por lo mismo, en Debug 1 no se comparan los bits 31:28 (copia de MSR[7:4]).	Las secuencias integradas suponen registros que guardan lo que se les escribe. Con estas exclusiones el bit bash y el access quedan sobre LCR, IER, DLL, DLM y scratch, que sí se comportan así.

13. Modelo de cobertura funcional

La cobertura funcional vive en uart_cov, un uvm_component con un uvm_analysis_imp por fuente. Las fuentes son los monitores y los eventos del modelo de referencia, nunca los drivers. Por ejemplo, el tipo de error de un carácter RX lo decide el monitor de srx midiendo la línea (ancho de bit real, bit de parada, paridad), no lo que el driver quiso mandar. Al final de cada episodio se imprime una línea [COV_CURVE] ep=<n> fcov=<%> para armar la curva de saturación de la línea base.

Covergroup	Dónde se muestrea	Coverpoints	Cruces	Bins ilegales o ignorados (justificación)
cg_line_cfg	Por cada carácter completo en stx (monitor TX) o en srx (monitor RX), con el formato vigente del modelo	cp_dir (TX, RX), cp_wls (5, 6, 7, 8), cp_stb (1, 1.5/2), cp_par (ninguna, impar, par, stick 1, stick 0), cp_loop, cp_dl_band (1, 2 a 16, 17 a 255, 256 a 4095, 4096 a 65535), cp_data (00, FF, 55, AA, otro)	wls × stb × par × dir; dir × loop; dl_band × dir	Ninguno
cg_dl	Monitor Wishbone, al completar la escritura del divisor (DLL o DLM)	cp_dl (0, 1, 2 a 16, 17 a 255, 256 a 4095, 4096 a 65534, 65535), cp_order (DLM primero, DLL primero)	dl × order	Ninguno
cg_fifo_ctrl	Monitor Wishbone al escribir FCR	cp_tl (1, 4, 8, 14), cp_rx_clr, cp_tx_clr, cp_traffic (reposo, TX activo, RX activo, ambos)	tl × rx_clr × tx_clr; clr × traffic	Ninguno
cg_tx	Modelo en cada escritura a THR, y monitor TX por carácter	cp_level (nivel de la FIFO TX antes de escribir: 0, 1, 2 a 7, 8 a 14, 15, 16 = escritura perdida), cp_break (break visto en stx)	level × wls	Ninguno

Covergroup	Dónde se muestrea	Coverpoints	Cruces	Bins ilegales o ignorados (justificación)
cg_rx	Monitor RX (srx) por carácter, modelo al empujar a la FIFO, y monitor Wishbone al leer RBR	cp_err (ninguno, paridad, trama, break, glitch descartado, glitch con falso inicio, baudios dentro, baudios fuera), cp_level (nivel de la FIFO RX al recibir: 0, bajo el TL, en el TL, 15, lleno = overrun), cp_data, cp_short_stop, cp_rd_empty (lectura de RBR con la FIFO vacía)	err × par; err × wls; level × tl	Error de paridad con paridad deshabilitada: ignorado, no hay bit de paridad que corromper
cg_err	Monitor Wishbone en cada lectura de LSR, con el valor leído	cp_oe, cp_pe, cp_fe, cp_bi, cp_fifo_err (bit 7), cp_dr, cp_thre, cp_temt, cp_tl	oe × tl; pe, fe y bi × wls; thre × temt	thre = 0 con temt = 1: ilegal, el transmisor no puede estar vacío con datos en la FIFO (si aparece es un error del DUT y se reporta)
cg_int	Monitor Wishbone en cada lectura de IIR y monitor de interrupción en cada flanco de int_o	cp_iir_id (ninguna, RLS, RDA, TIMEOUT, THRE, MSI), cp_ier (16 valores), cp_sw_mode (interrupciones, sondeo), cp_multi (fuentes pendientes según el modelo: 0, 1, 2, 3 o más), cp_int_edge (sube, baja)	iir_id × sw_mode; iir_id × multi; iir_id × tl; iir_id × ier	Un tipo de interrupción con su bit de IER en 0: ignorado, está enmascarada y no puede aparecer
cg_modem	Monitor de módem en cada cambio de entrada; monitor Wishbone al leer MSR y al escribir MCR	cp_sig (CTS, DSR, RI, DCD), cp_edge (sube, baja), cp_loop, cp_delta (DCTS, DDSR, TERI, DDCD al leer MSR), cp_teri (TERI con RI subiendo y con RI bajando, AMB-04), cp_out (DTR, RTS, OUT1, OUT2 al escribir MCR)	sig × edge × loop; out × loop	Ninguno
cg_reg_access	Monitor Wishbone en cada transacción	cp_addr (0 a 7, 8, 12), cp_rw, cp_dlab, cp_sel (un bit en cada carril, 1111, 0000, dos bits; BAD = 1111 en registros de 8 bits, 0000 o dos bits)	addr × rw × dlab (incluye LSR, MSR e IIR con DLAB = 1, AMB-07); sel × rw	Escrituras a FCR (2) y MCR (4) con DLAB = 1: ignorado, las secuencias reconfiguran siempre con DLAB = 0
cg_reset	Monitor Wishbone (ve wb_rst_i) con el estado del modelo	cp_when (reposo, TX activo, RX activo, ambos, durante un acceso al bus), cp_width (1, 2 a 5 ciclos)	when × width	Ninguno
cg_reconfig	Monitor Wishbone al escribir LCR, DLL, DLM, FCR o MCR con tráfico activo según el modelo	cp_reg (LCR, DL, FCR, MCR), cp_traffic (TX activo, RX activo, ambos)	reg × traffic	Ninguno

14. Cobertura de código

Métrica	Jerarquía incluida	Meta	Exclusiones previstas y justificación
Línea	tb_top.dut (uart_top y sus submódulos), con -cm_hier "+tree tb_top.dut"; el ambiente no cuenta	≥ 95 %	Ninguna prevista. Si aparece código muerto en la línea base, se excluye con un .el y su justificación
Condición	tb_top.dut (uart_top y sus submódulos), con -cm_hier "+tree tb_top.dut"; el ambiente no cuenta	≥ 95 %	Candidatas: condiciones que dependen de combinaciones imposibles de los estados de las FSM. Se revisan una por una en la línea base
Rama	tb_top.dut (uart_top y sus submódulos), con -cm_hier "+tree tb_top.dut"; el ambiente no cuenta	≥ 95 %	Candidatas: ramas default de case que ya cubren todos los valores. Las default de wb_sel_i sí son alcanzables (AMB-05) y no se excluyen
FSM	Máquinas tstate (tb_top.dut.regs.transmitter) y rstate (tb_top.dut.regs.receiver)	≥ 95 %	Candidatas: transiciones que la herramienta extrae pero que el código no permite. Se justifican con el código del RTL
Toggle	tb_top.dut (uart_top y sus submódulos), con -cm_hier "+tree tb_top.dut"; el ambiente no cuenta	≥ 95 %	Candidatos: bits que no cambian en el modo de 32 bits (por ejemplo bits altos de wb_dat_o en lecturas de 8 bits que no dependen del carril). La lógica de DATA_BUS_WIDTH_8 y baud_o no se compila, así que no entra en la cuenta

15. Criterios de cierre

Criterio	Meta	Cómo se mide
Cobertura funcional	≥ 95 %	Reporte de URG de la regresión completa fusionada, sobre los covergroups de la sección 13
Cada métrica de cobertura de código	≥ 95 %	Reporte de URG sobre uart_top y sus instancias, para línea, condición, rama, FSM y toggle
UVM_ERROR en la regresión	0, salvo errores del DUT documentados	Resumen de UVM de cada corrida (UVM_ERROR y UVM_FATAL); los errores del DUT confirmados se listan en el informe con semilla y plusargs

Criterio	Meta	Cómo se mide
Aserciones con cover alcanzado	100 %	Reporte de URG: cada cover property de la sección 11 con al menos un éxito
Línea base de la prueba base	350 semillas, con el ambiente validado antes con 40 semillas sin errores falsos	Curva de cobertura funcional y de código contra el número de semillas

16. Riesgos y mitigación

Riesgo	Impacto	Mitigación
Regresión demasiado lenta por valores grandes de DL	Alto: pocas semillas corridas y cierre de cobertura en riesgo	Concentrar la aleatorización en DL de 1 a 16 (95 %), limitar los caracteres por episodio con DL ≥ 256, dejar la cola de DL con baja probabilidad y plusarg, y correr las 350 semillas en paralelo
Candidatos a error del DUT (AMB-03, AMB-04) que no se confirman, o que se confirman tarde	Medio: errores falsos en el scoreboard o filas de la tabla de ambigüedades (sección 3.7) incorrectas	Confirmarlos en simulación en las primeras semanas; el scoreboard sigue la especificación y cuenta los errores conocidos del DUT sin tratarlos como errores del ambiente; la aserción A-MDM-01 se reporta con la misma severidad (+UART_SCB_DUT_BUG_SEV)
Comportamientos que la especificación no define (wb_sel_i ilegal, DLAB = 1, retardo de THRE)	Medio: el scoreboard marca errores falsos	Modelar lo que hace el RTL y documentar las tolerancias en la sección 10
No alcanzar 95 % en alguna métrica de código (toggle, condición, FSM)	Alto: baja la nota de cobertura	Empezar la línea base temprano, clasificar cada hueco, cerrar los alcanzables con archivos .args y justificar las exclusiones en archivos .el
Nombres inconsistentes entre secciones y componentes (objetos de configuración, plusargs, covergroups)	Medio: el plan se contradice y el ambiente cuesta integrar	Usar la tabla de plusargs (sección 7) y los objetos de configuración (sección 6.5) como única fuente de nombres, y hacer revisión cruzada de cada sección
Que un integrante no entienda partes que escribió otro	Alto: la nota individual es cero si no aprueba la entrevista	Revisión cruzada obligatoria, cada integrante lee y explica las partes de los demás antes de cada entrega
Problemas con el entorno de VCS en el servidor (variables, licencias, disponibilidad)	Medio: se pierden sesiones de trabajo	Dejar documentado el comando de entorno (alias en ~/.bashrc), guardar los logs de compilación y trabajar el código fuera del servidor cuando se pueda
Diferencia entre la especificación y el RTL en Debug 2 (AMB-08)	Medio: uvm_reg_hw_reset_seq fallaría en Debug 2 si el RAL siguiera el layout de la especificación	El RAL usa el layout y el reset del RTL (MCR en 21:17, FCR en 23:22, 32'h00C00000) y la diferencia queda documentada en AMB-08

17. Distribución del trabajo

Cada integrante es responsable principal de algunos componentes, pero los tres deben entender todo el ambiente (entrevista individual, sección 8.2 del enunciado). S1, S2... son semanas del proyecto.

Componente o tarea	Responsable principal	Revisor (otro integrante)	Semana
Plan de pruebas (E1): secciones 1 a 5, 15 a 17	Wilbert Román	Gerald Blanco	S1 y S2
Plan de pruebas (E1): secciones 6 a 8 y 13	Gerald Blanco	Leonardo Pérez	S1 y S2
Plan de pruebas (E1): secciones 9 a 12 y 14	Leonardo Pérez	Wilbert Román	S1 y S2
Objetos de configuración, parse_plusargs() y print_knobs()	Gerald Blanco	Leonardo Pérez	S3
Agente Wishbone y monitor de interrupción	Leonardo Pérez	Gerald Blanco	S3
Agente serie (driver con errores, monitores de stx y srx)	Wilbert Román	Gerald Blanco	S3
RAL: reg_block, adaptador, predictor, puerta trasera y hw_reset_seq	Leonardo Pérez	Wilbert Román	S3 y S4
Secuencias y secuencia virtual principal	Gerald Blanco	Wilbert Román	S3 y S4
Modelo de referencia y scoreboard TX y RX (E2)	Wilbert Román	Leonardo Pérez	S4
Agente de módem	Wilbert Román	Leonardo Pérez	S4
Aserciones SVA	Leonardo Pérez	Gerald Blanco	S4
Scoreboard de LSR, IIR, MSR e int_o	Leonardo Pérez	Wilbert Román	S5
Cobertura funcional (uart_cov) y curva [COV_CURVE]	Gerald Blanco	Wilbert Román	S5
Script de regresión (semillas en paralelo), fusión con urg y línea base	Wilbert Román	Gerald Blanco	S5
Análisis de huecos, archivos .args y exclusiones .el	Los tres	Los tres	S6 en adelante
Informe final y presentación (E3)	Los tres	Los tres	Semana 18